

第一章 数据库系统概论

(第一章只考一个5分的分析题，也不会太偏向问答，而是去年的那种分析)

(侧重于理解记忆，而不是背诵，当然也需要一定程度上的记忆)

1.1 数据库

数据库是一个**依照特定数据模型**有组织的数据集合，**旨在存储、管理和检索信息**。它可以被视为一个电子文件柜，**允许用户以结构化的方式**

1.什么是数据库：**存储和访问数据**。数据库通常由**数据库管理系统（DBMS）**进行管理，**DBMS**是用于创建、维护和操作数据库的软件。

2.数据库的主要作用：

- 数据存储与管理（高效存储、数据组织）
 - 数据检索与查询（快速查询、复杂查询支持）
 - 数据完整性与一致性（约束机制、事务管理）
 - 数据安全性（权限管理、数据加密与备份）
 - 数据共享与协作（多用户访问、数据集成）
-

1.2 数据模型

3.什么是数据模型

4.数据模型的主要组成部分

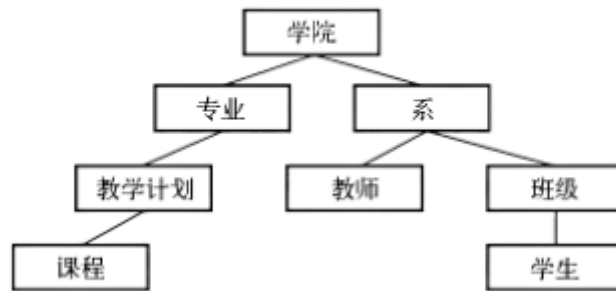
- 数据结构
- 数据操作
- 数据约束

5.数据库使用的几种经典数据模型

- 层次模型
- 网状模型
- 关系模型

6.层次模型

- 以**树形结构**组织数据、数据以记录形式存储、每个记录可以有多个子记录
- 适合表示**一对多关系**
- 查询效率高，但灵活性差



7.网状模型

- 记录之间可以是**多对多关系**、以**图结构**组织数据、记录通过指针互相连接
- 能够表示更加复杂的数据关系
- 实现和管理相对复杂

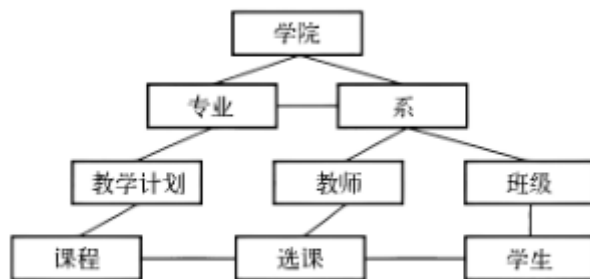


图 1-2 高校教务系统的网状数据模型

8.关系模型

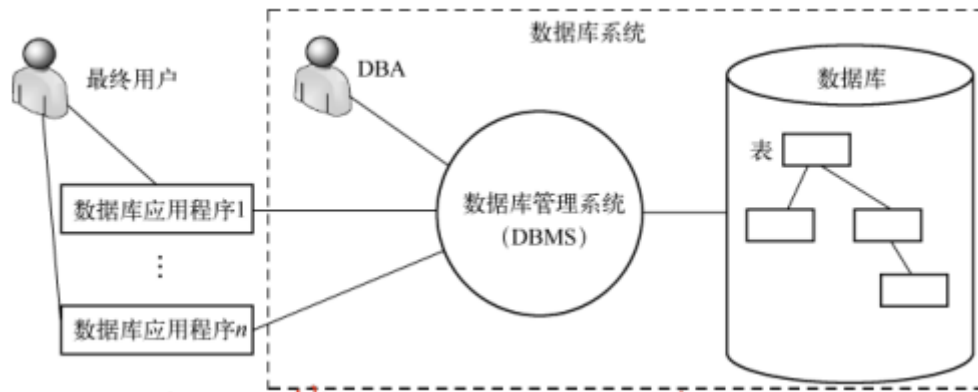
- 使用行和列表示数据、数据以表格（关系）的形式组织
- 支持一对一、一对多、多对多的关系
- 使用SQL语言进行数据操作，简化了数据访问

教师信息表				课程信息表			
工号	姓名	职称	学院	课程号	课程名称	学时	学分
2001	刘东	讲师	计算机	001	数据库原理	64	4
2002	王崎	教授	软件工程	002	程序设计	48	3
2003	姜力	副教授	软件工程	003	数据结构	48	3

开课目录表			
工号	课程号	开课学期	最多人数
2001	002	春季	100
2002	001	秋季	120
2003	003	秋季	100

1.3数据库系统

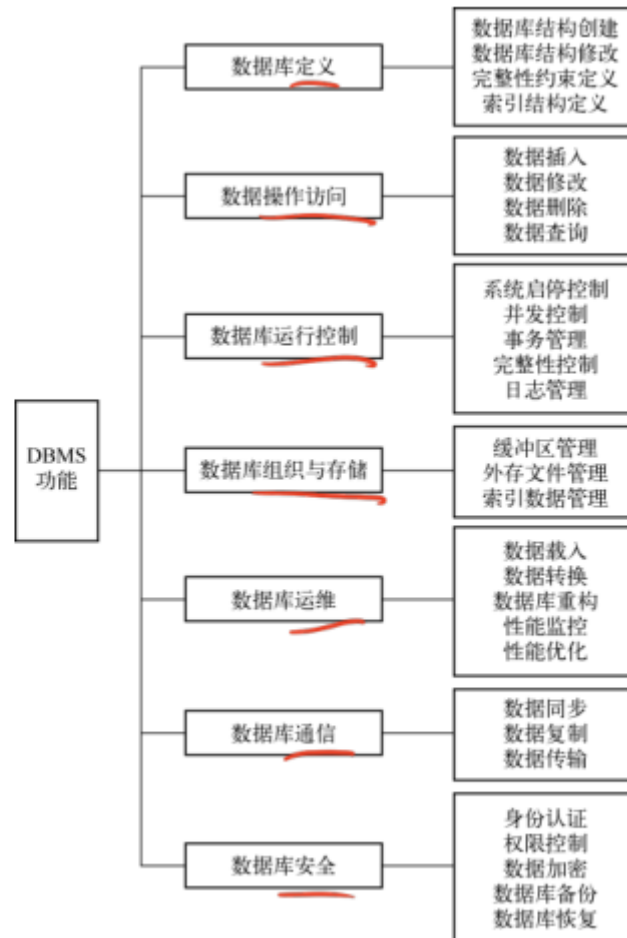
1.数据库系统由五部分构成：数据库、数据库管理系统（DBMS）、应用程序、数据库管理员、数据库用户



- 数据库：
- 数据库管理系统：
- 应用程序：
- 数据库管理员：
- 数据库用户：

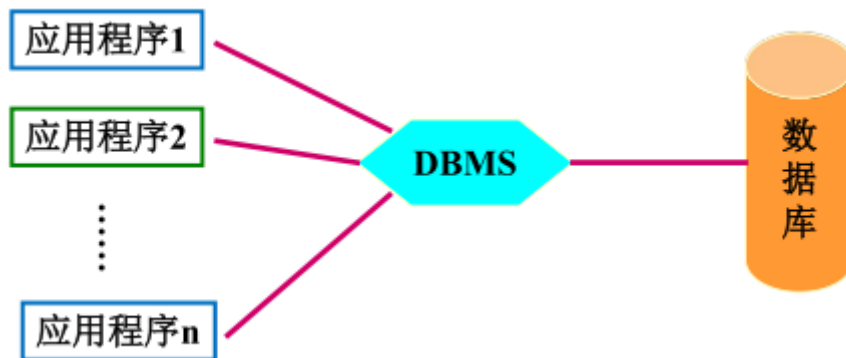
2.数据库管理系统的基本功能

数据库管理系统基本功能

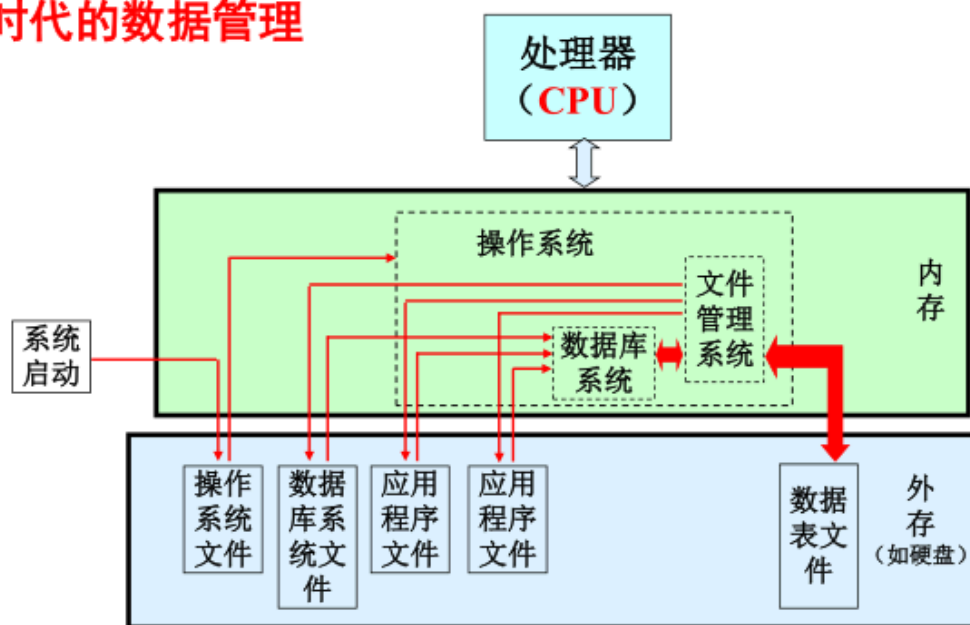


1.4数据管理的发展阶段

- 1.人工管理数据阶段（数据和程序一一对应）
- 2.文件系统阶段（有外存设备、有文件系统）
- 3.数据库系统阶段（把数据统一管理、控制、共享使用）



数据库时代的数据管理



4.数据库管理相对文件管理的优点：

- 应用程序与数据相互独立，解决了应用程序对数据文件的依赖问题
- 应用访问数据库时使用标准的数据库操作语言，编程访问简单
- 数据组织结构化，共享性高，冗余小
- 提供数据的安全访问机制，并保证数据的完整性、一致性、正确性

1.5数据库技术的发展演进

1.第一代数据库技术（以层次模型和网状模型为主）

2.第二代数据库技术（引入了关系数据库模型）

3.第三代数据库技术（面相对象数据库、对象关系数据库；旨在解决传统关系型数据库在处理复杂数据类型和关系时的局限性）

4.第四代数据库技术（NoSQL数据库、新型数据存储解决方案）

- NoSQL数据库技术：不遵循严格的关系模型，目标是处理大数据、支持分布式存储和高并发访问的应用场景
- 新型数据存储解决方案：分布式数据库、云数据库、实时数据处理平台

1.6数据库领域技术

- 1.NewSQL数据库：具备高并发处理能力和水平扩展性，适合大规模分布式环境
- 2.领域数据库：针对特定领域的数据库系统
- 3.数据仓库与数据挖掘
- 4.商业智能（BI）
- 5.大数据分析

1.7数据库应用系统的类型

- 1.业务处理系统（支持和管理企业内部各项业务流程的系统，如订单处理、库存管理.....）
- 2.管理信息系统（组织、存储和管理机构各个部门的数据，如人员管理、物资管理、资金管理.....）
- 3.决策支持系统（帮助管理者和决策者在复杂的决策过程中进行分析和选择）

1.8数据库应用系统的结构

1.客户/服务器架构（C/S架构）

- 应用程序分为两个主要组件：客户端、服务器
- 客户端负责用户界面和用户交互
- 服务器负责业务逻辑和数据访问
- 适用于中小型系统

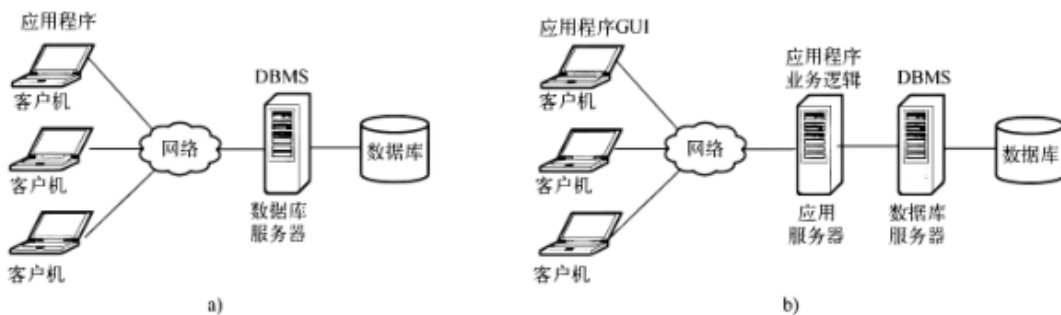


图 1-7 客户/服务器架构
a) 2层 C/S 架构 b) 3层 C/S 架构

2.浏览器/服务器架构（B/S架构）

- 客户端是Web浏览器，通过HTTP协议与服务器通信
- 服务器端负责处理业务逻辑和数据访问，并将动态生成的Web页面发给客户端进行展示
- 适用于Internet和Intranet环境下的应用系统

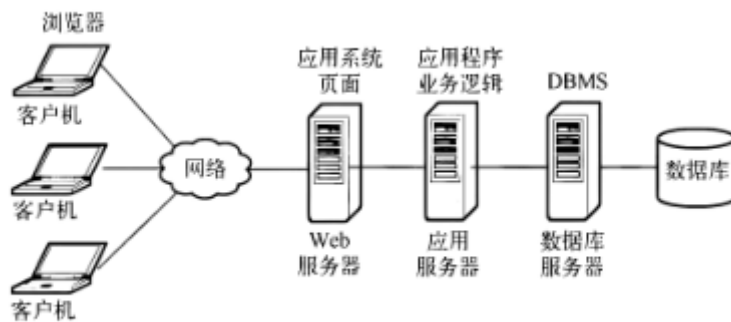


图 1-8 浏览器/服务器架构

3.分布式架构

- 将数据库应用系统的不同组件分布在多个物理或逻辑节点上
- 需要考虑数据一致性、通信协议和负载均衡等问题
- 适用于大规模应用系统和分布式环境

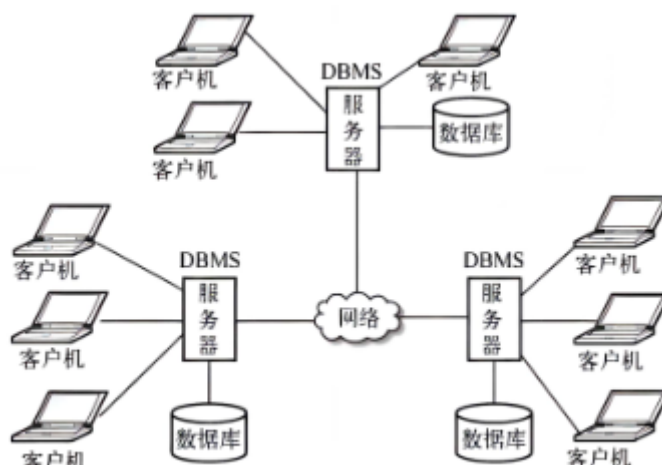


图 1-9 分布式架构

1.9数据库应用系统的生命周期

- 1.系统需求分析
- 2.系统数据架构设计（概念设计、逻辑设计、物理设计）
- 3.系统数据库实现
- 4.系统测试
- 5.系统运行与维护

1.10数据库管理系统软件的分类型

- 1.关系型数据库管理系统（RDBMS）
- 2.非关系型数据库管理系统（NoSQL DBMS）（适合处理大规模非结构化数据）
- 3.内存数据库管理系统（IMDBMS）（将数据存储在内存在中，可以快速响应、高并发）
- 4.分布式数据库管理系统（DDBMS）（管理分布在多个物理或逻辑节点上的数据库的系统）
- 5.数据仓库管理系统（DWMS）（管理和分析大规模历史数据的数据库管理系统）
- 6.对象数据库管理系统（ODBMS）（管理和操作面向对象数据的数据库管理系统）

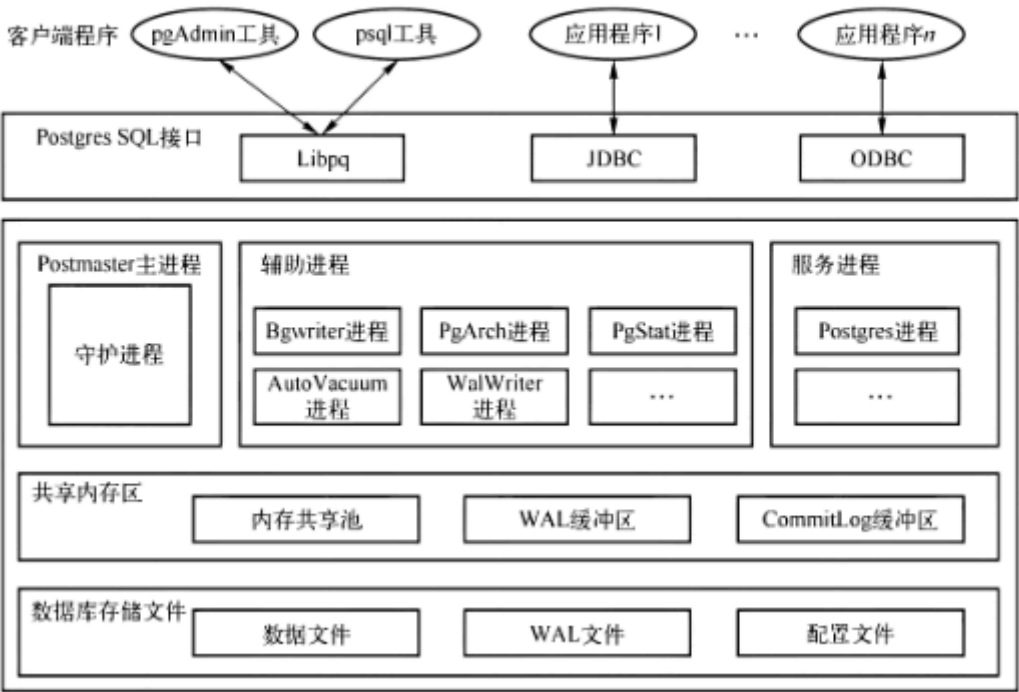
1.11主流开源数据库管理系统

MySQL、Postgresql

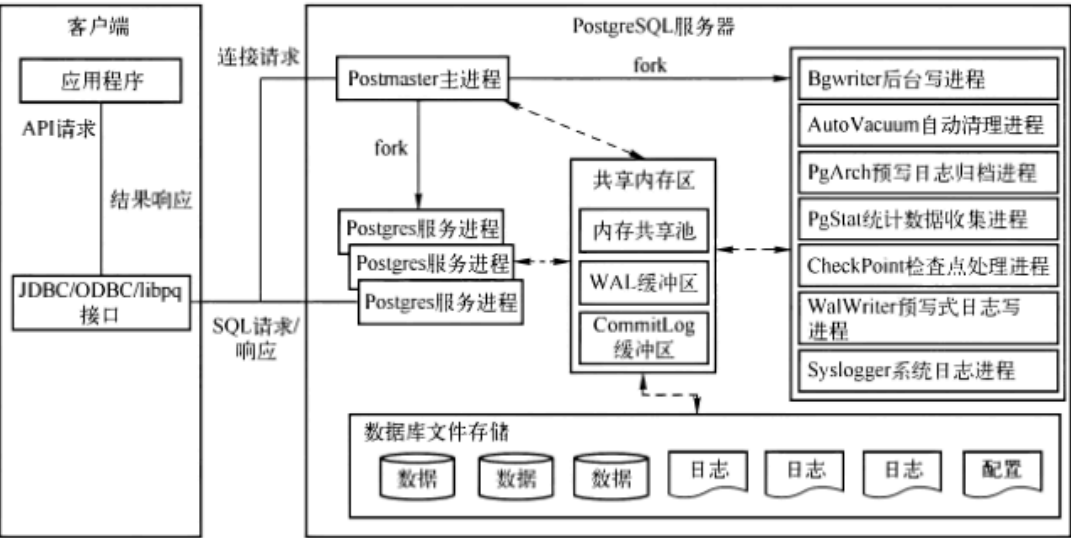
MariaDB、MongoDB、Redis

1.12Postgresql数据库管理系统

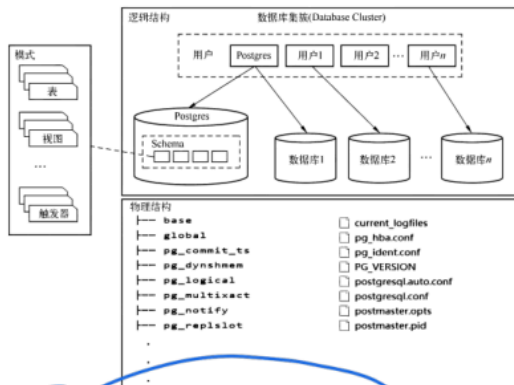
(1) 系统总体架构



(2) 系统运行架构



(3) 数据库逻辑结构和物理结构



去年考了那个图

上次问 base 目录是干嘛的

子目录	用途	子目录	用途
base	存储每个数据库对应子目录下的各类数据文件	pg_stat_tmp	存储统计信息子系统临时文件
global	存储全局的系统表数据文件	pg_subtrans	存储子事务状态数据文件
pg_commit_ts	存储事务提交时间戳数据文件	pg_tblspc	存储表空间数据文件
pg_dynshmem	存储被动态共享所使用的数据文件	pg_twophase	存储预备事务状态文件
pg_logical	存储用于逻辑复制的状态数据文件	pg_wal	存储预写日志文件
pg_multixact	存储多事务状态的数据文件	pg_xact	存储事务提交状态数据文件
pg_notify	存储 LISTEN/NOTIFY 状态数据文件	PG_VERSION	PostgreSQL 主版本号文件
pg_repslot	存储复制槽数据文件	pg_hba.conf	客户端认证控制文件
pg_serial	存储已提交的序列化信息数据文件	pg_ident.conf	用户名映射文件
pg_snapshots	存储快照数据文件	postgresql.conf	参数配置文件
pg_stat	存储统计信息数据文件	postgresql.auto.conf	ALTER SYSTEM 参数配置文件
		postmaster.opts	记录服务器端上一次启动的命令选项文件

表 1-3 PostgreSQL 数据库 data 目录下的配置文件

文件	用途
PG_VERSION	PostgreSQL 主版本号文件
pg_hba.conf	客户端认证控制文件
pg_ident.conf	用户名映射文件
postgresql.conf	参数配置文件
postgresql.auto.conf	ALTER SYSTEM 参数配置文件
postmaster.opts	记录服务器端上一次启动的命令选项文件

第二章 关系数据模型

2.1 关系数据模型、笛卡尔积

1. 关系数据模型:

关系数据模型

采用二维表格结构来表示**实体和实体之间的联系**。在用户观点下，关系模型中数据的逻辑结构是一张二维表，它由行和列组成。

主键或码		学生的基本信息				关系名
		学号	姓名	住址	性别	兴趣爱好
关系		20060101	张江	04-201	男	排球
		20060102	魏明	04-203	男	足球
		20060103	王昆	05-102	女	羽毛球
		20060104	程香	05-102	女	羽毛球
		20060105	刘鹏	04-405	男	游泳
		20060106	王德启	04-203	男	排球
		20060107	武飞	04-205	男	篮球
		20060108	刘用	04-102	男	篮球
		20060109	程文	05-304	女	乒乓球

关系模式：学生(学号、姓名、住址、性别、兴趣爱好)

2. 主要术语:

- ① 元组(tuple): 表中的一行
- ② 属性(attribute): 表中的一列, 列名即为属性名
- ③ 域(domain): 属性的取值范围
- ④ 分量(element): 元组中的一个属性值
- ⑤ 候选键(candidate key): 在关系中能唯一标识元组、没有多余属性的属性集
- ⑥ 主键(primary Key): 用户选作元组标识的一个候选键称为主键
- ⑦ 关系(relation): 关系是多个元组的集合, 且是规范化的二维表格。
- ⑧ 关系模式(relation mode): 用关系名 (属性名1, 属性名2,, 属性名n) 来表示称为关系模式

学生信息 (学号 身份证号 性别 年龄 身高 体重 宿舍号)

- (学号)、(身份证号) 都是候选键, 可以唯一标识一个元组, 且没有多余属性
- 从候选键选出来的一个键就是主键

3.笛卡尔积

设 $D_1, D_2, \dots, D_n$ 是 n 个域, 则它们的笛卡尔积为

$$D_1 \times D_2 \times \dots \times D_n = \{(d_1, d_2, \dots, d_n) \mid d_i \in D_i, i=1,2,\dots,n\}$$

其中每一个元素称为一个 n 元组(n -tuple), 简称元组;

元组中的每个值 d_i 称为一个分量(component).

4.数学意义上的关系

数学意义上的关系 (Relation)

李四 语文

笛卡尔积 $D_1 \times D_2 \times \dots \times D_n$ 的子集合,

记作 $R (D_1, D_2, \dots, D_n)$

关系名

n 为关系的目或元

(2) 数学意义的关系可以是无限个元组的集合。

数学意义的关系不适合数据库的实际应用, 必须做如下限制:

- (1) 数据库关系模型中的关系必须是有限的元组集合
- (2) 数据库关系模型中的属性列表的顺序是可交换的, 允许任意顺序; 具有数据库中关系满足交换律。

2.2关系的性质、类型

- 1.每个属性 (列) 都是不可再分解的, 列中不能包含子列
- 2.列的个数和每列的数据类型是固定的 (即每一列中的分量是同类型、来自同一值域的)
- 3.每个属性要给予不同的属性名

4.列的顺序是无关紧要的

5.元组不可以重复（即一个关系中任意两个元组不能完全相同，不能有相同的行）

6.行的顺序是无关紧要的

关系的三种类型：

1.基本关系：客观存在的基本表

2.查询表：由基本表按照一定的条件检索得到的结果

3.视图：从一个或多个基本关系上导出的关系。不对应实际的存储数据，是一个虚关系

2.3关系操作

1.种类：

- 查询操作：选择、投影、连接、除、并、交、差
- 维护操作：增加、删除、更新

2.关系模型的三类完整性：

- 实体完整性约束（元组唯一、主键不为空；由主键实现实体完整性）
- 参照完整性约束（外键，值，必须等于所参照的表里面的对应列的某一个值，或者为空）
- 用户定义的完整性（数据类型约束、非空、唯一、默认值、检查.....）

2.4关系代数运算

RDBS（关系型数据库）的数据操纵语言：关系代数

1.关系代数的运算对象是关系，运算结果也为关系。其运算按运算符的不同可分为两类：

- 传统的集合运算
- 专门的关系运算

2.4.1传统的集合运算

1、并（Union）： $R \cup S = \{t \mid t \in R \vee t \in S\}$

2、交（Intersection）： $R \cap S = \{t \mid t \in R \wedge t \in S\}$

3、差（Difference）： $R - S = \{t \mid t \in R \wedge t \notin S\}$

4、笛卡尔积（广义）： $R \times S = \{\widehat{t_r t_s} \mid t_r \in R \wedge t_s \in S\}$

（两个分别为n目和m目的关系R和S的笛卡尔积是一个(n + m)列的元组的集合。

元组的前n列是关系R的一个元组，后m列是关系S的一个元组。）

2.4.2专门的关系运算

注意符号

选择、投影、连接（条件链接、等值连接、自然连接、外连接）、除

1.选择 and 投影

1、选择 (Selection)，又称限制 (Restriction) 从行的角度的运算

$\sigma_F(R)$: 在关系R中选出满足条件F的元组形成新的关系。

条件表达式

2、投影 (Projection) 从列的角度的运算

$\pi_A(R)$: 在R中选出若干属性列组成一个新关系。

属性组

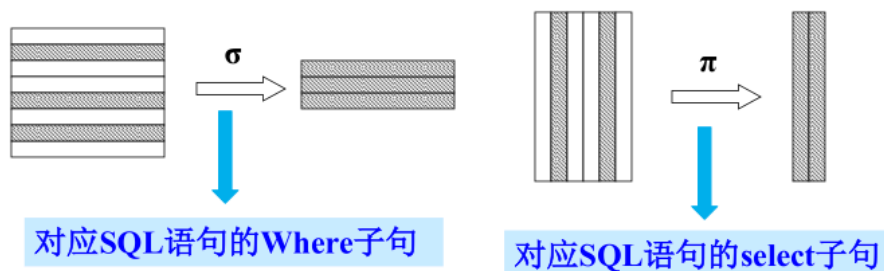
投影后若有重复行，则自动保留一个

共同点：为一元关系操作符。

选择：从关系实例中选择出满足条件的行。操作符： σ

投影：从关系实例中抽出所需的一列或多列。操作符： π

条件表达式中的比较操作符： $>$ ， $>=$ ， $<$ ， $<=$ ， $=$ ， $\neq$



例子：

例1：查询信息系（IS系）全体学生

$\sigma_{Sdept = 'IS'}(Student)$ 或 $\sigma_5 = 'IS'(Student)$

例2：查询年龄小于20岁的学生

$\sigma_{Sage < 20}(Student)$ 或 $\sigma_4 < 20(Student)$

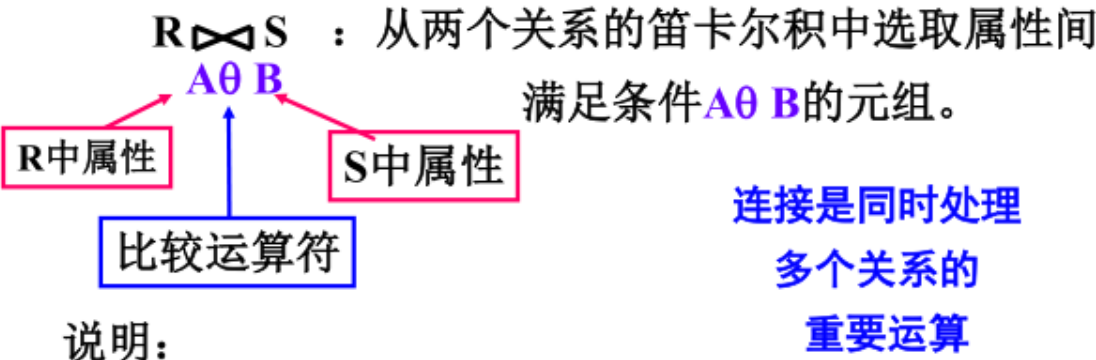
例3：查询学生的姓名和所在系

即求Student关系上学生姓名和所在系两个属性上的投影

$\pi_{Sname, Sdept}(Student)$ 或 $\pi_{2, 5}(Student)$

2.连接

1.定义:



2. $R \bowtie_{A \theta B} S = \sigma_{A \theta B} (R \times S)$

3.当 θ 为等号, 且A,B两属性相同时, 称为自然连接, 记为 $R \bowtie S$

4.若仅有 θ 为等号的条件, 称为等值连接

5.自然连接是等值连接的特殊情况

6.自然连接将去掉重复属性

(种类: 条件连接、等值连接、自然连接、外连接)

1.条件连接:

概念: $R \bowtie_c S = \sigma_c(R \times S)$

说明: 条件c会用到R和S的属性, 如R.name, R.i(位置)。

R			S		$R \bowtie_{C < E} S$				
A	B	C	B	E	A	R.B	C	S.B	E
a_1	b_1	5	b_1	3	a_1	b_1	5	b_2	7
a_1	b_2	6	b_2	7	a_1	b_1	5	b_3	10
a_2	b_3	8	b_3	10	a_1	b_2	6	b_2	7
a_2	b_4	12	b_3	2	a_1	b_2	6	b_3	10
			b_5	2	a_2	b_3	8	b_3	10

2.等值连接:

(2)等值连接 (Equijoin)

概念：是条件连接的特例，即连接条件由等式组成，如： $R.name1=S.name2$

从关系R与S的广义笛卡尔积中选取A、B属性值相等的那些元组，即等值连接为：

示例： $R \bowtie_{R.B=S.B} S$

R			S	
A	B	C	B	E
a ₁	b ₁	5	b ₁	3
a ₁	b ₂	6	b ₂	7
a ₂	b ₃	8	b ₃	10
a ₂	b ₄	12	b ₃	2
			b ₅	2

A	R.B	C	S.B	E
a ₁	b ₁	5	b ₁	3
a ₁	b ₂	6	b ₂	7
a ₂	b ₃	8	b ₃	10
a ₂	b ₃	8	b ₃	2

3.自然连接：

(3) Natural Join(自然连接)

概念：是等连接的特例 即：等式中参与连接的字段名相同，这时可忽略连接条件，即为： $R \bowtie S$

示例： $R \bowtie S$

R			S	
A	B	C	B	E
a ₁	b ₁	5	b ₁	3
a ₁	b ₂	6	b ₂	7
a ₂	b ₃	8	b ₃	10
a ₂	b ₄	12	b ₃	2
			b ₅	2

A	B	C	E
a ₁	b ₁	5	3
a ₁	b ₂	6	7
a ₂	b ₃	8	10
a ₂	b ₃	8	2

注意：自然连接这里去掉了重复的属性（列），可以和上面等值连接对比一下，两者连接所用的条件是一样的，但结果不同

4.外连接（自然连接的特例）

概念：涉及有空值的自然连接，是自然连接的特例。

说明：自然连接是寻找相同字段值相等的行。但如果一个关系中的该字段在另一关系中没有相等值的行，自然连接不会显示该行，而外连接则将以NULL值形式显示该行。

外连接的种类：

- ① 左外连接 (LEFT OUTER JOIN)
- ② 右外连接 (RIGHT OUTER JOIN)
- ③ 全外连接 (FULL OUTER JOIN)

说明：

① 与外连接对应，前面三种连接为**内连接(Inner Join)**；

② 关系代数中没有外连接的描述，但SQL标准中外连接查询语句；

左外连接：对于 $R \bowtie S$ ，如果在S中没有匹配R的行，则以NULL值表示，最后的结果是以左边的关系R为准，即左边关系中的所有行均应出现在结果中，如果在S中没有对应的行，则以NULL表示之。

R	sid	sname	age	grade	S	sid	cid	score	结果:	sid	cid
	8	何大明	19	2		8	101	91		8	101
	11	李 峰	20	3		35	106	84		11	null
	35	陈 胜	21	4						35	106

左外连接示意图

右外连接：对于 $R \bowtie S$ ，如果在R中没有匹配S的行，则以NULL值表示，最后的结果以右边的关系S为准。

R	sid	sname	age	grade	S	sid	cid	score	结果:	sid	cid	sname
	8	何大明	19	2		8	101	91		8	101	何大明
	11	李 峰	20	3		35	106	84		35	106	陈 胜
	35	陈 胜	21	4		66	119	88		66	119	null

右外连接示意图

全外连接：对于 $R \bowtie S$ ，没有匹配的R和S的行，也都出现于结果中。

R	sid	sname	age	grade	S	sid	cid	score	结果:	sid	cid	sname
	8	何大明	19	2		8	101	91		8	101	何大明
	11	李 峰	20	3		35	106	84		11	null	李 峰
	35	陈 胜	21	4		66	119	88		35	106	陈 胜
										66	119	null

全外连接示意图

3.除

第三章 数据库SQL语言

(对于本章，仅在编程题中考四个5分题：表创建、表查询、表统计、视图定义；)

(当然，其他章节可能也会牵扯到隐含考察这一章的)

(注意，考存储过程、触发器的时候大概率会考到update、delete)

3.1 一些总结

1.某些数据库管理系统对SQL语句进行了再开发和扩展：Postgre SQL扩展为PL/PGSQL；

2.SQL语言的类型：DDL、DML、DCL、TPL、CCL

DDL：

```
create database
alter database
create table
alter table
drop table
create index
drop index
```

DML：

```
select
update
delete
insert
```

DCL（数据控制语言）

```
grant
deny
revoke
```

TPL（事务处理语言）

```
begin transaction
commit
rollback
```

CCL（游标控制语言）

```
declare cursor
fetch into
close cursor
```

3.核心的9个动词：

语言简洁，易学易用，核心功能只需9个动词

① DDL: CREATE、DROP、ALTER

② DML: SELECT、INSERT、UPDATE、DELETE

③ DCL: GRANT、REVOKE

3.2数据库修改SQL

语句基本格式： **ALTER DATABASE** <数据库名> <修改内容>;

修改数据库的名称：**ALTER DATABASE** old_database_name **RENAME TO** new_database_name;

修改数据库的所有者：**ALTER DATABASE** database_name **OWNER TO** new_owner;

2、删除数据库

① **DROP DATABASE IF EXISTS** database_name;

3.3创建数据库表

3.3.1.核心语法结构

```
CREATE TABLE table_name (  
    column1 data_type [constraints],  
    column2 data_type [constraints],  
    ...  
    [table_constraints]  
);
```

- ① **table_name**: 要创建的表的名称。
- ② **column1, column2, ...**: 表中的列名。
- ③ **data_type**: 列的数据类型，例如：INTEGER、VARCHAR(n)、DATE 等。
- ④ **constraints**: 可选的约束条件，例如：NOT NULL、UNIQUE、PRIMARY KEY 等。
- ⑤ **table_constraints**: 可选的表级约束，例如：PRIMARY KEY、FOREIGN KEY 等。

3.3.2完整性约束关键词

- primary key——主键
- not null——非空
- null——空
- unique——值唯一
- check——有效性检查
- default——缺省值

3.3.3数据类型

1.数值类型：

- ① **INTEGER**: 整型, 4 字节。
- ② **BIGINT**: 大整型, 8 字节。
- ③ **SMALLINT**: 小整型, 2 字节。
- ④ **DECIMAL(p, s)**或**NUMERIC(p, s)**: 精确小数, p 为总位数, s 为小数位数。
- ⑤ **REAL**: 4 字节, 单精度浮点数。
- ⑥ **DOUBLE PRECISION**: 8 字节, 双精度浮点数。

2. 字符类型:

- ① **VARCHAR(n)**: 变长字符串, 最大长度为 n。
- ② **CHAR(n)**: 定长字符串, 长度为 n。
- ③ **TEXT**: 可变长度字符串, 没有长度限制。

3. 日期和时间类型:

- ① **DATE**: 日期数据, 格式为 YYYY-MM-DD, 范围从 4713 BC 到 5874897 AD。
- ② **TIME**: 时间数据, 格式为 HH:MM:SS, 可以带时区。
- ③ **TIMESTAMP**: 时间戳数据, 格式为 YYYY-MM-DD HH:MM:SS。
- ④ **INTERVAL**: 表示时间间隔, 支持多种单位 (如天、小时、分钟等)。

4. 自增长类型:

在 PostgreSQL 中, **SERIAL** 是一种用于创建自增长整数列的特殊数据类型。它通常用于表的主键, 以确保每一行都有一个唯一的标识符。使用 **SERIAL** 类型时, PostgreSQL 会自动为新插入的行生成一个唯一的整数值。提供了三种不同大小的 **SERIAL** 类型:

- ① **SERIAL**: 对应于 **INTEGER**, 范围是从 1 到 2,147,483,647。
- ② **BIGSERIAL**: 对应于 **BIGINT**, 范围是从 1 到 9,223,372,036,854,775,807。
- ③ **SMALLSERIAL**: 对应于 **SMALLINT**, 范围是从 1 到 32,767。

5. 布尔类型:

BOOLEAN: 存储布尔值, 取值为 **TRUE**、**FALSE** 或 **NULL**

6. 地理空间类型:

- ① POINT: 表示二维空间中的点。
- ② LINE: 表示无限长的直线。
- ③ LSEG: 表示线段。
- ④ BOX: 表示矩形。
- ⑤ POLYGON: 表示多边形。
- ⑥ PATH: 表示路径（开或闭）。
- ⑦ CIRCLE: 表示圆。

3.3.4例子

```
CREATE TABLE classes (
    class_id SERIAL PRIMARY KEY,           --自增长字段，主键
    class_code VARCHAR(20) UNIQUE NOT NULL, --班级编码，唯一
    class_name VARCHAR(50) NOT NULL,       --班级名称，非空
    major VARCHAR(50) NOT NULL             --专业信息
);

CREATE TABLE students (
    student_id SERIAL PRIMARY KEY,         --自增长字段，主键
    student_number VARCHAR(20) UNIQUE NOT NULL, --学号，唯一
    student_name VARCHAR(50) NOT NULL,     --姓名，非空
    gender CHAR(1) CHECK (gender IN ('M', 'F')) DEFAULT 'M', --性别，M=男，F=女，缺省值为M
    class_id INT,
    CONSTRAINT classid_fk FOREIGN KEY (class_id) REFERENCES classes(class_id)
);

CREATE TABLE enrollments (
    student_id INT,                        --学生ID
    course_class_id INT,                  --课程班ID
    score DECIMAL(5, 2),                  --考试分数
    CONSTRAINT enrollment_pk PRIMARY KEY (student_id, course_class_id), --定义复合主键
    CONSTRAINT sid_fk FOREIGN KEY (student_id) REFERENCES students(student_id), --定义学生ID外键约束
    CONSTRAINT course_cid_fk FOREIGN KEY (course_class_id) REFERENCES course_classes(course_class_id)
    --定义课程班ID外键约束
);
```

3.4修改数据库表结构

ALTER TABLE <表名> <修改方式>;

ALTER TABLE teachers **ADD COLUMN** title **VARCHAR(50);**

- add, 增加新列或者列的完整性约束
- drop, 删除指定列或者列的完整性约束条件
- rename, 修改表名、列名
- alter, 修改列的数据类型

3.5删除表结构

DROP TABLE <表名>;

注意: 该语句将删除该表的所有数据及其结构

3.6向表中插入记录

INSERT INTO table_name (column1, column2, column3, ...)

VALUES (value1, value2, value3, ...);

INSERT INTO table_name (column1, column2, column3, ...)

VALUES

(value1a, value2a, value3a, ...),

(value1b, value2b, value3b, ...),

(value1c, value2c, value3c, ...);

注意: 可以在 **VALUES** 子句中使用逗号分隔的多个值列表来插入多行数据。

如果要插入的所有列都有值, 可以省略列名。值的顺序必须与表中列的顺序一致。

如果某个列有默认值, 可以在插入时省略该列, PostgreSQL 将使用默认值。

3.7 SQL内置函数类型

1.数学函数 (lx的PPT没有, 鉴定为不考)

① **ABS(x)**: 返回 x 的绝对值。

SELECT ABS(-10); -- 返回 10

② **CEIL(x)** 或 **CEILING(x)**: 返回大于或等于 x 的最小整数。

SELECT CEIL(10.5); -- 返回 11

③ **FLOOR(x)**: 返回小于或等于 x 的最大整数。

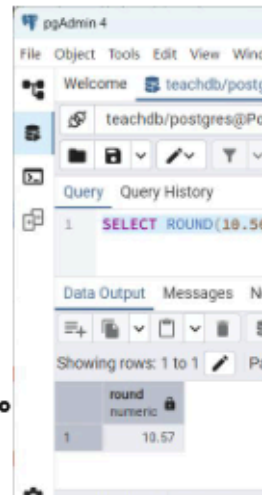
SELECT FLOOR(10.5); -- 返回 10

④ **ROUND(x, d)**: 将 x 四舍五入到 d 位小数。

SELECT ROUND(10.567, 2); -- 返回 10.57

⑤ **RANDOM()**: 返回一个 0 到 1 之间的随机数。

SELECT RANDOM(); -- 返回随机数



2. 字符串函数 (lx的PPT没有, 鉴定为不考)

① **LENGTH(string)**: 返回字符串的长度。

SELECT LENGTH('Hello'); -- 返回 5

② **UPPER(string)**: 将字符串转换为大写。

SELECT UPPER('hello'); -- 返回 'HELLO'

③ **LOWER(string)**: 将字符串转换为小写。

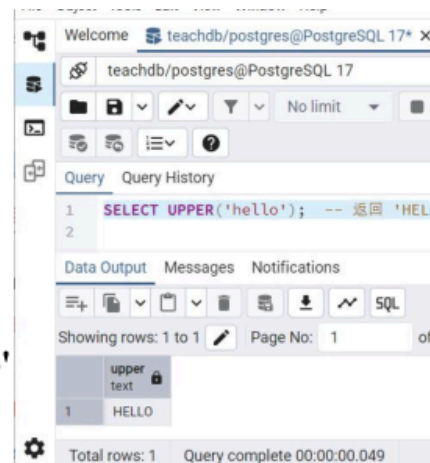
SELECT LOWER('HELLO'); -- 返回 'hello'

④ **TRIM(string)**: 去除字符串两端的空格。

SELECT TRIM(' Hello '); -- 返回 'Hello'

⑤ **SUBSTRING(string FROM start FOR length)**: 返回字符串的子串。

SELECT SUBSTRING('Hello World' FROM 1 FOR 5); -- 返回 'Hello'



3. 日期和时间函数 (lx的PPT没有, 鉴定为不考)

① **CURRENT_DATE**: 返回当前日期。

SELECT CURRENT_DATE; -- 返回当前日期

② **CURRENT_TIME**: 返回当前时间。

SELECT CURRENT_TIME; -- 返回当前时间

③ **NOW()**: 返回当前的日期和时间。

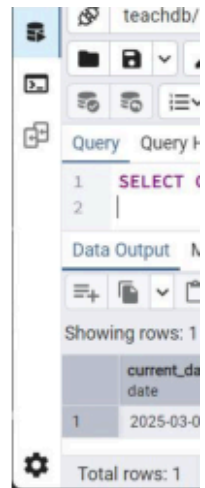
SELECT NOW(); -- 返回当前的日期和时间

④ **AGE(timestamp)**: 返回当前时间与给定时间之间的间隔

SELECT AGE(NOW(), '2020-01-01'); -- 返回时间间隔

⑤ **DATE_PART(field, source)**: 提取日期或时间的特定部分。

SELECT DATE_PART('year', NOW()); -- 返回当前年份



4.聚合函数 (考统计应该会用到)

① **COUNT(column)**: 返回指定列的行数。

SELECT COUNT(student_id) FROM students; -- 返回学生数量

② **SUM(column)**: 返回指定列的总和。

SELECT SUM(score) FROM enrollments; -- 返回成绩总和

③ **AVG(column)**: 返回指定列的平均值。

SELECT AVG(score) FROM enrollments; -- 返回平均成绩

④ **MAX(column)**: 返回指定列的最大值

SELECT MAX(score) FROM enrollments; -- 返回最高成绩

⑤ **MIN(column)**: 返回指定列的最小值。

SELECT MIN(score) FROM enrollments; -- 返回最低成绩

5.条件函数 (lx的PPT没有, 鉴定为不考)

① COALESCE(value1, value2, ...): 返回第一个非空值。

```
SELECT COALESCE(NULL, NULL, 'Hello'); -- 返回 'Hello'
```

② NULLIF(value1, value2): 如果 value1 等于 value2, 则返回 NULL, 否则返回 value1。

```
SELECT NULLIF(10, 10); -- 返回 NULL
```

③ CASE: 用于条件判断。

```
SELECT
  CASE
    WHEN score >= 90 THEN '优秀'
    WHEN score >= 80 THEN '良好'
    ELSE '及格'
  END AS performance
FROM enrollments;
```

6. 类型转化函数 (lx的PPT没有, 鉴定为不考)

① CAST(expression AS type): 将表达式转换为指定类型。

```
SELECT CAST('123' AS INTEGER); -- 返回 123
```

② ::type: 另一种类型转换的语法。

```
SELECT '123'::INTEGER; -- 返回 123
```

3.8 SQL查询语句

```
SELECT column1, column2, ...
FROM table_name
WHERE condition
ORDER BY column1, column2, ...
GROUP BY column1, column2, ...
HAVING condition
LIMIT number
OFFSET number;
```

- ① SELECT: 指定要查询的列。
- ② FROM: 指定要查询的表。
- ③ WHERE: 用于过滤结果集, 指定条件。
- ④ ORDER BY: 用于对结果集进行排序。
- ⑤ GROUP BY: 用于将结果集按一个或多个列进行分组。
- ⑥ HAVING: 用于过滤分组后的结果集。
- ⑦ LIMIT: 限制返回的行数。
- ⑧ OFFSET: 用于跳过结果集中的前几行。

3.8.1. order by 排序

```
SELECT * FROM students ORDER BY student_name ASC;
```

--该查询将按学生姓名的升序排列结果。

```
SELECT * FROM students ORDER BY class_id DESC;
```

--该查询将按班级 ID 的降序排列结果。

3.8.2.使用limit、offset

SELECT * FROM students **LIMIT 5**; --该查询将返回 students 表中的前 5 行记录。

SELECT * FROM students **LIMIT 5 OFFSET 2**; -- 该查询将跳过前 2 行，返回接下来的 5 行记录。

3.8.3使用join连接多个表

该查询将返回每个学生的姓名及其所选课程的名称。

```
SELECT s.student_name, c.course_name
FROM students s
JOIN enrollments e ON s.student_id = e.student_id
JOIN course_classes cc ON e.course_class_id = cc.course_class_id
JOIN courses c ON cc.course_id = c.course_id;
```

该查询将返回每个学生的姓名及其所选课程
的名称，也可以使用下列查询：

```
SELECT s.student_name, c.course_name
FROM students s,
      enrollments e,
      course_classes cc,
      courses c
where s.student_id = e.student_id and
      e.course_class_id = cc.course_class_id and
      cc.course_id = c.course_id;
```

3.8.4使用distinct去重

```
SELECT DISTINCT gender FROM students;
--该查询返回 students 表中所有不同的性别。
```

3.8.5使用子查询

① 子查询返回单个值

```
SELECT student_name  
FROM students  
WHERE class_id = (  
    SELECT class_id  
    FROM classes  
    WHERE class_code = 'CS101');
```

--该查询将返回在 CS101 班级中的所有学生姓名。

② 子查询返回多个值

```
SELECT student_name  
FROM students  
WHERE class_id IN (  
    SELECT class_id FROM classes  
    WHERE major = '计算机科学');
```

--该查询将返回所有计算机科学专业的学生姓名。

3.8.6使用union合并多个查询结果

```
SELECT student_name AS name, '学生' AS role
```

```
FROM students
```

```
UNION
```

```
SELECT teacher_name AS name, '教师' AS role
```

```
FROM teachers;
```

--该查询将返回一个包含所有学生和教师姓名的列表，并标识他们的角色。

这种用法：标注学生

3.8.7外连接

外连接（Outer Join）是 SQL 中的一种连接操作，用于从两个或多个表中检索数据，即使在某些表中没有匹配的记录。外连接有三种主要类型：左外连接（LEFT JOIN）、右外连接（RIGHT JOIN）和全外连接（FULL OUTER JOIN）。

1.左外连接：

左外连接返回左表（第一个表）中的所有记录，以及右表（第二个表）中匹配的记录。如果右表中没有匹配的记录，则结果中右表的字段将返回 NULL。语法格式：

```
SELECT columns
```

```
FROM left_table
```

```
LEFT JOIN right_table ON left_table.column = right_table.column;
```

例如：编写SQL语句使用左外连接查询所有老师的排课情况，即使某些课程没有排课，包括老师姓名、课程名、开课学期及学时数。

```
SELECT t.teacher_name, c.course_name, cc.semester, c.hours
```

```
FROM courses c
```

```
LEFT JOIN course_classes cc ON cc.course_id = c.course_id
```

```
LEFT JOIN teachers t ON t.teacher_id = cc.teacher_id
```

```
ORDER BY t.teacher_name;
```

2.右外连接：

右外连接返回右表中的所有记录，以及左表中匹配的记录。如果左表中没有匹配的记录，则结果中左表的字段将返回 NULL。

```
SELECT columns  
FROM left_table  
RIGHT JOIN right_table ON left_table.column = right_table.column;
```

编写SQL语句使用右外连接查询所有老师的排课情况，即使某些老师没有排课，包括老师姓名、课程名、开课学期及时数。

```
SELECT t.teacher_name, c.course_name, cc.semester, c.hours  
FROM courses c  
right JOIN course_classes cc ON cc.course_id = c.course_id  
right JOIN teachers t ON t.teacher_id = cc.teacher_id  
ORDER BY t.teacher_name;
```

3. 全外连接:

全外连接返回左表和右表中的所有记录。如果在左表中没有匹配的记录，则左表的字段将返回 NULL；如果在右表中没有匹配的记录，则右表的字段将返回 NULL。

```
SELECT columns  
FROM left_table  
FULL OUTER JOIN right_table ON left_table.column = right_table.column;
```

编写SQL语句使用全外连接查询所有老师和所有课程的排课情况，即使某些老师没有排课，或者某些课程没有排课，都需要显示。包括老师姓名、课程名、开课学期及时数。

```
SELECT t.teacher_name, c.course_name, cc.semester, c.hours  
FROM courses c  
FULL OUTER JOIN course_classes cc ON cc.course_id = c.course_id  
FULL OUTER JOIN teachers t ON t.teacher_id = cc.teacher_id  
ORDER BY t.teacher_name;
```

PPT上的例子的表: (用于P112的题目练习)

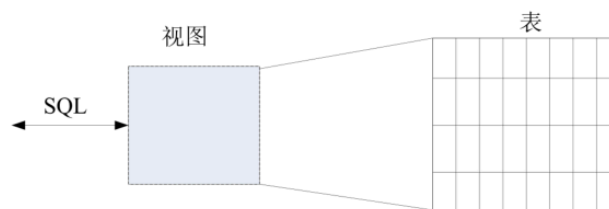
- 班级表, classes (class_id, class_code, class_name, major)
- 学生信息表, students(student_id, student_number, student_name, gender, class_id)
- 教师表, teachers(teacher_id, employee_number, teacher_name, gender)
- 课程表, courses(course_id, course_code, course_name, hours, credits)

- 授课班级表, course_classes(course_class_id, course_id, teacher_id, semester)
- 选课表, enrollments(student_id, course_class_id, score)

注意：有些字段，题目中没明说要加，但查询结果应该有，就比如“查询每门课程的学生人数和最高成绩”，虽然没说，但肯定要加上课程名称字段吧？

3.9数据库视图

视图是一个虚拟表，它是基于 SQL 查询结果的命名查询。视图并不存储数据，而是存储查询的 SQL 语句。当你查询视图时，PostgreSQL 会动态执行视图定义中的 SQL 语句，并返回结果。视图可以简化复杂的查询，提供数据的安全性和抽象层。



(2) 创建视图的 SQL 语法

```
CREATE VIEW view_name AS
SELECT column1, column2, ...
FROM table_name
WHERE condition;
```

① view_name: 视图的名称。

② SELECT: 定义视图的查询语句，可以是任何有效的 SELECT 查询。

(3) 创建视图示例

创建一个视图，显示每个学生的姓名、课程名称、教师姓名和成绩。

```
CREATE VIEW student_course_info AS
SELECT s.student_name, c.course_name, t.teacher_name, e.score
FROM students s
JOIN enrollments e ON s.student_id = e.student_id
JOIN course_classes cc ON e.course_class_id = cc.course_class_id
JOIN courses c ON cc.course_id = c.course_id
JOIN teachers t ON cc.teacher_id = t.teacher_id;
```

在该视图上执行查询：

```
select * from student_course_info;
```

如果需要删除视图，可以使用以下 SQL 语句：

```
DROP VIEW student_course_info;
```

3.10 数据库索引

（4）创建索引的 SQL 语法

```
CREATE INDEX index_name ON table_name (column1, column2, ...);
```

- ① index_name：索引的名称。
- ② table_name：要创建索引的表的名称。
- ③ column1, column2, ...：要索引的列。

```
CREATE INDEX idx_student_name ON students (student_name);
```

随着数据的变化，索引可能会变得不够高效，使用 REINDEX 命令重建索引：

```
REINDEX INDEX idx_student_name;
```

如果需要删除索引，可以使用以下 SQL 语句：

```
DROP INDEX idx_student_name;
```

3.11 删除表的数据

1.使用delete语句

DELETE 语句用于根据指定条件删除表中的一行或多行数据。

其基本语法如下：

```
DELETE FROM table_name
```

```
WHERE condition;
```

- ① table_name：要删除数据的表名。
- ② condition：用于指定要删除的行的条件。如果没有指定条件，则会删除表中的所有行。

2.使用truncate语句

TRUNCATE 语句用于快速删除表中的所有数据，而不记录每一行的删除操作。
其基本语法如下：

TRUNCATE TABLE table_name;

table_name: 要清空数据的表名。

TRUNCATE 的特点

- ① **快速**: TRUNCATE 通常比 DELETE 更快，因为它不逐行删除数据，而是直接释放数据页。
- ② **不可回滚**: 在大多数情况下，TRUNCATE 是不可回滚的，意味着一旦执行，数据将无法恢复。
不支持数据库事务
- ③ **不触发触发器**: TRUNCATE 不会触发 DELETE 触发器。

3.12 数据修改

UPDATE table_name

SET column1 = value1, column2 = value2, ...

WHERE condition;

- ① table_name: 要更新数据的表名。
- ② SET: 指定要更新的列及其新值。
- ③ WHERE: 可选部分，用于指定要更新的行。如果没有 WHERE 子句，表中的所有行都会被更新。

UPDATE students

SET class_id = 2

WHERE student_name = '张三';

UPDATE students

SET class_id = 2, student_number = '20230004'

WHERE student_name = '李四';

第四章 数据库设计与规范化

4.1 E-R模型

4.1.1 模型基本元素

- 三种基本元素：实体、属性、联系
- 1.实体：
- 可以是人、物品、机构单位.....
 - 符号：两栏矩形方框，上栏写实体的名字，下栏写各个属性名
- 2.属性：
- 标识符：能够唯一标识不同实体实例的属性或者属性集
 - 复合标识符：找不到单个属性可以作为标识符，选取多个属性的组合作为标识符
 - 作为表示符的属性需要带有下列线
- 3.联系：
- 一般用动词来命名
 - 用带有形似鸟足的连线符号表示实体之间的联系

4.1.2 实体联系的类型

1.按多重性分类

- 一对一
- 一对多
- 多对多

2.按参与性分类

- 可选的联系
- 强制的联系

不同二元联系的符号表示：

联系符号	含 义
	实体联系为可选，最小基数为 0，最大基数为 1
	实体联系为强制，最小基数为 1，最大基数为 1
	实体联系为可选，最小基数为 0，最大基数为 N
	实体联系为强制，最小基数为 1，最大基数为 N

3.按继承性分类

第一种分法：

- 互斥继承联系：父实体中的某个实例只能属于某个子实体，不能同时属于多个子实体
- 非互斥继承联系：父实体的一个实例可以属于多个子实体

第二种分法：

- 完整继承联系：父实体中的实例完整地各个子实体分别继承
- 非完整继承联系：没有完整被各个子实体分别继承

符号：

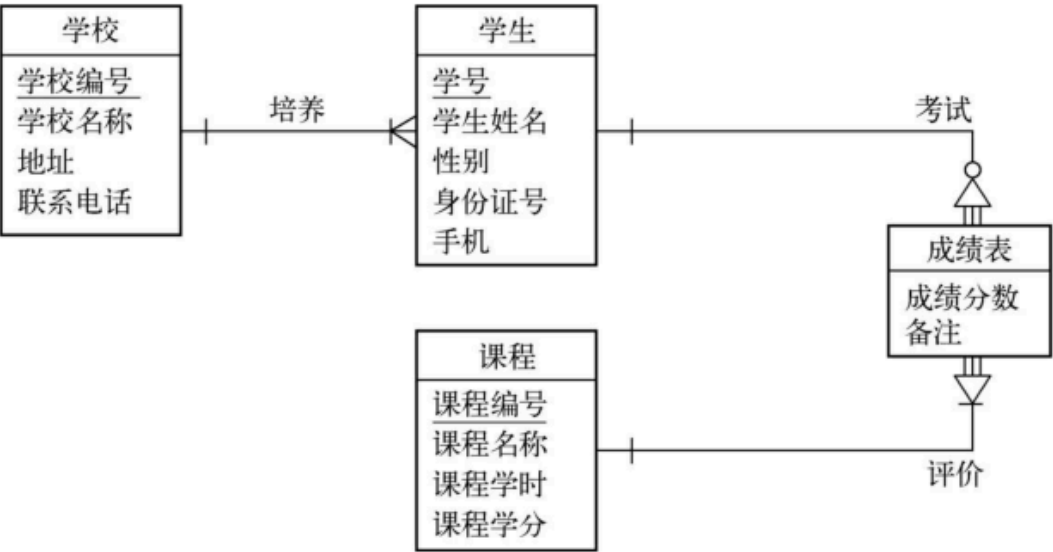
继承联系类型	模型符号
非互斥继承联系	
互斥继承联系	
完整继承联系	
非完整继承联系	

4.1.3强弱实体

- 一个实体的存在必须以另一个实体的存在为前提
- ER模型中， 给出一个弱实体时必须给出它所依赖的强实体

4.1.4标识符依赖实体

- 标识符依赖实体：弱实体的标识符含有所依赖实体的标识符
- 非标识符依赖实体：弱实体有自己的标识符， 没有把所依赖实体的标识符作为自己标识符的组成部分
- 标识符依赖弱实体和非标识符依赖弱实体的联系连线图形符号是不同的
 - 标识符依赖弱实体， 在弱实体一侧有一个三角形的符号
 - 非标识符依赖实体， 仅为基本的鸟足符号



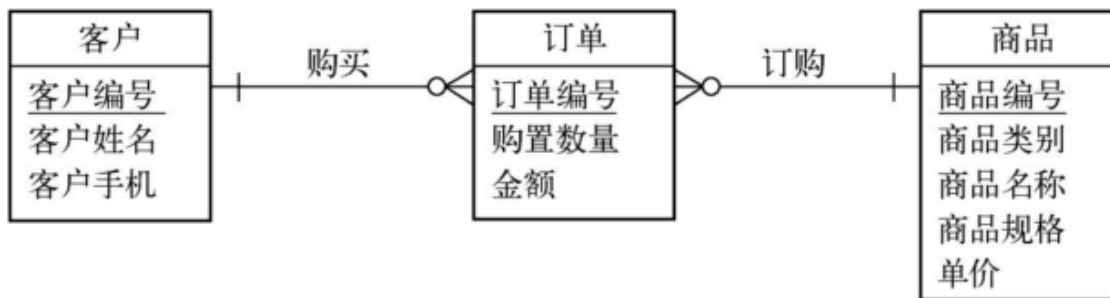


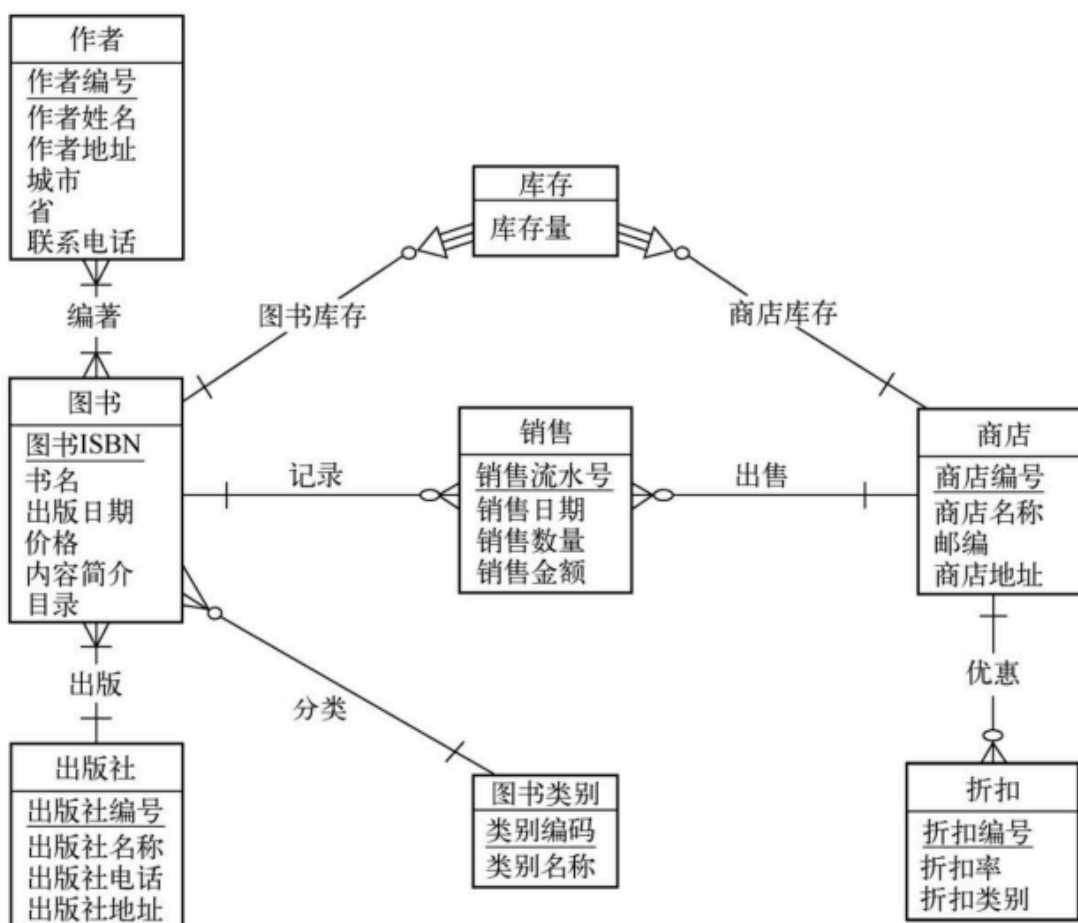
图4-15 非标识符依赖弱实体示例

4.2数据库建模设计

4.2.1概念数据模型设计

步骤：

- 1.抽取与标识实体
- 2.分析与标识实体联系（需要给出联系的名称，联系名称不能重复）
- 3.定义实体属性与标识符（标识符值唯一、不能为空）



4.2.2逻辑数据模型设计

- 概念数据模型在系统设计角度的延伸
- 既要体现数据库实现的数据模型针对性（如关系数据库设计），又要不依赖于具体的DBMS产品

与CDM区别：

- CDM标识符不区分，且只画下划线
- LDM标识符分为主键标识符、外键标识符，画下划线的同时要写<pi> <fi> <pi>下面也要加下划线

CDM到LDM的转换：

- 1.多对多转换成一对多
- 2.增加外键标识符（一对多，一的主键加到多）
- 3.画上主键标识符、外键标识符

（思路，先看看哪里有多对多，记得加一个新的实体；

然后写每个实体的时候，看看实体身上有几个“多”，写属性时候记得加上外键）

- 在LDM的设计中，一项重要的工作就是对数据模型进行规范化与完善设计

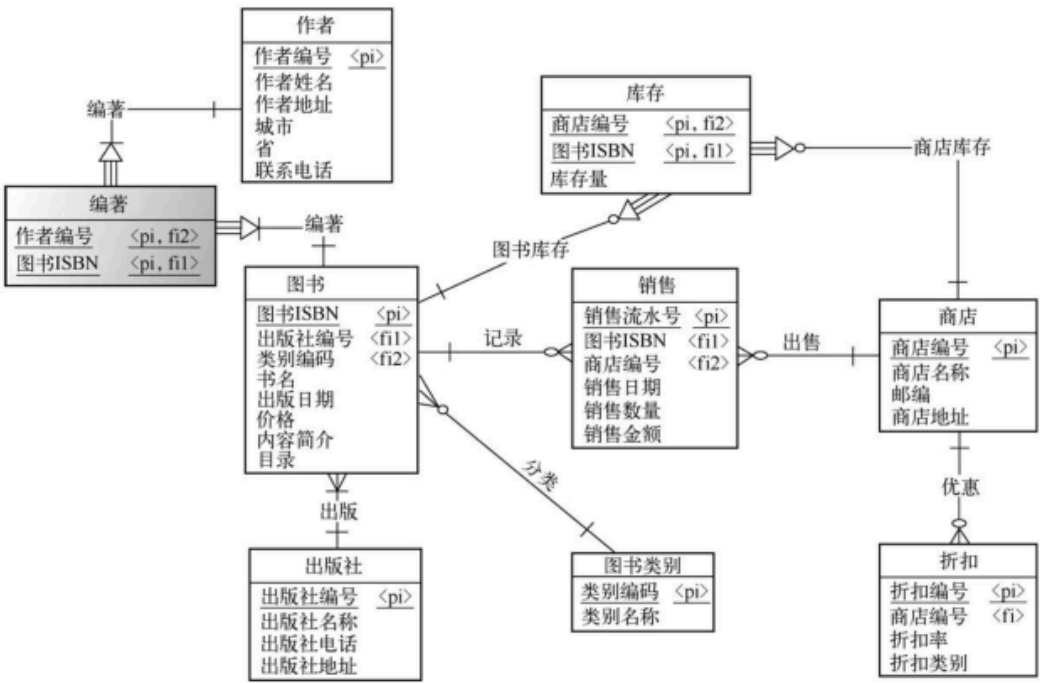


图4-21 初步的图书销售管理系统逻辑数据模型

4.2.3物理数据模型设计

- 不再使用ER图
- 采用关系模型图元素符号（关系表、参照完整性约束、主键、外键、视图、触发器、存储过程等）

设计步骤：

1. 将E-R模型图中每一个实体对应转换成一个关系表，实体属性转换为对应表的列，实体标识符转换为对应表的主键
2. 将实体联系转换为关系表之间的、主、外键关系，并定义表之间的参照完整性约束
3. 扩展视图、索引、存储过程及触发器等数据库对象

ER模型转换为关系模型的各类处理：

1. 实体到关系表的转换：

- 每个实体定义一个关系表
- 实体的属性转换为表中的列
- 主键标识符转化为主键、外键标识符转换为外键
- 需要为每个列定义数据类型、空值状态等

2. 弱实体到关系表的转换

- 非标识符依赖弱实体：弱实体的表中加入强实体的标识符作为外键列
- 标识符依赖弱实体：弱实体的表中加入强实体的标识符作为外键列，同时作为主键列

3. 实体联系的转换

(1) 1:1 实体联系的转换

- 将其中一个表的主键放入另一个表作为外键

(2) 1: N 实体联系的转换

- 1端关系表的主键放入N端关系表中作为外键
- 如果存在标识符依赖弱实体，则1端关系表的主键放入N端关系表中作为外键，同时也作为主键

(3) M: N 实体联系的转换

- 增加一个关联表
- 新的表共有两个列，分别来自两个表的主键
- 这两个列构成符合主键，同时也是外键

(4) 实体继承联系的转换

- 父表主键的属性放到子表，既作外键，又作主键

(5) 实体递归联系的转换

- 在表中加入一个外键属性参照本表主键属性

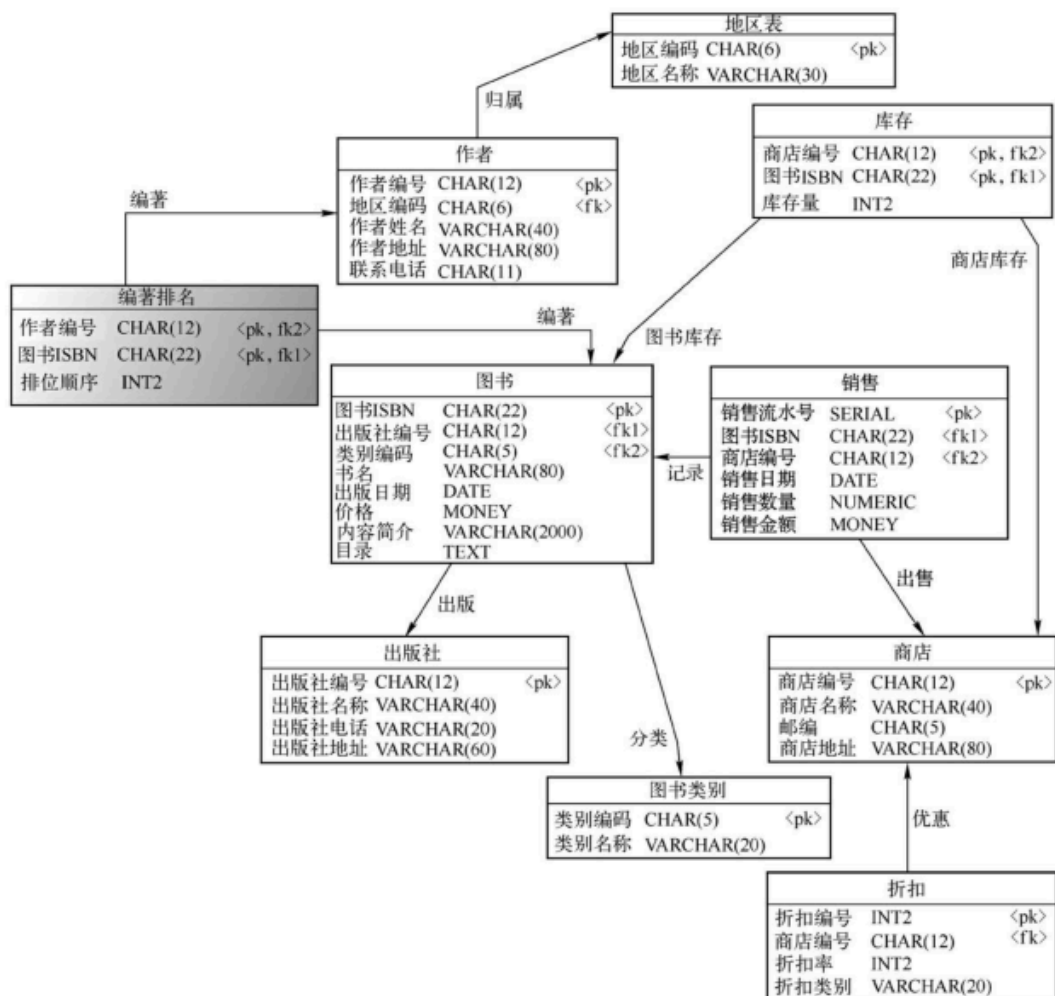


图4-41 图书销售管理系统物理数据模型设计结果

4.3 数据库规范化设计

不规范可能出现的异常：

- 插入异常
- 删除异常
- 冗余
- 更新异常

4.3.1 函数依赖理论

1. 函数依赖

3、函数依赖

函数依赖是属性之间的约束关系。

定义：设X、Y是关系表R的属性（组），如果对于R的所有元组都有：X的每一具体值都只有一个Y的值与之对应，则称X函数决定Y，或Y函数依赖于X，记作 $X \rightarrow Y$ 。

换句话说，如果知道了X的值，就可以在表中确定与之对应的Y的值（只有一个）。

- 若Y不函数依赖于X，记作 $X \nrightarrow Y$ 。
- 若 $X \rightarrow Y$ ，且 $Y \rightarrow X$ ，记作 $X \leftrightarrow Y$ 。

函数依赖关心的问题：是一个或一组属性的值决定其他属性的值。

如果X、Y是1:1的联系，则 $X \leftrightarrow Y$ 。

如学号 $\leftrightarrow$ 联系电话，即知道了学号，就可以在表中确定其联系电话；同样地，知道了联系电话，也可以在表中确定其学号。

如果X、Y是n:1的联系，则 $X \rightarrow Y$ 。

如学号 $\rightarrow$ 班号，即知道了学号，就可以在表中确定其班号；相反地，如果知道了班号，却无法确定学号。

如果X、Y是m:n的联系，则X和Y不存在函数依赖关系。

如果学号和课程号没有函数依赖关系。即知道了学号，无法在表中确定课程号；同样地，如果知道了课程号，也无法确定学号。

2.平凡函数依赖和非平凡函数依赖

定义：设X、Y是关系表R的属性（组），且 $X \rightarrow Y$ ，若 $Y \subseteq X$ ，则称为平凡依赖，否则称为非平凡依赖。

3.完全函数依赖和部分函数依赖

定义：设X、Y是关系表R的属性（组），且 $X \rightarrow Y$ ，若X存在某个子集 X_1 ，使 $X_1 \rightarrow Y$ 成立，则称Y部分依赖于X，否则称Y完全依赖于X。

如（学号，姓名） $\rightarrow$ 班号，而学号 $\rightarrow$ 班号，因此，班号部分依赖于（学号，姓名）。
（注：X中任何一个属性少了都不能确定Y）

（最小属性集）

4.传递函数依赖和非传递函数依赖

定义：设X、Y、Z是关系表R的属性（组），若 $X \rightarrow Y$ ， $Y \rightarrow Z$ ，且 $Y \nrightarrow X$ ，Y不属于X，则称X传递决定Z，或Z传递依赖于X，否则称Z非传递依赖于X。

如学号 $\rightarrow$ 班号，班号 $\rightarrow$ 学院，因此，学院传递依赖于学号，或学号传递决定学院。

学号	姓名	班号	学院	课程号	课程名	联系电话
14101	张三	99141	自动化	A101	高等数学	136xxxxxxxx
14101	张三	99141	自动化	A204	计算机网络	136xxxxxxxx
14122	李四	99141	自动化	A101	高等数学	130xxxxxxxx
14213	王五	99142	自动化	A101	高等数学	138xxxxxxxx
08113	赵六	99081	计算机	A107	大学物理	137xxxxxxxx
08218	钱七	99082	计算机	A204	计算机网络	133xxxxxxxx

- 如果Y能推出X，那么X和Y就是一一对应的，那么X就能直接推出来Z，就不存在传递依赖了

5.多值依赖

定义1:

多值依赖 (MultiValued Dependency, 缩写为MVD)

设 $R(U)$ 是属性集 U 上的关系模式， X, Y, Z 是 U 的子集，且 $Z = U - X - Y$ ，多值依赖 $X \twoheadrightarrow Y$ 成立当且仅当对 $R(U)$ 的任一关系 r ，任给的一对 (x, z) 值有一组 Y 的值，这组值仅仅取决于 x 值而与 z 值无关，称 X 多值决定 Y 或 Y 多值依赖于 X 。

定义2:

【定义 1.3】在架构设计和教学实践中，以平实的描述给出多值依赖的定义。

设 $R(U)$ 是属性集 U 上的一个关系模式。 X, Y, Z 是 U 的子集，并且 $Z = U - X - Y$ 。关系模式 $R(U)$ 中多值依赖 $X \twoheadrightarrow Y$ 成立，当且仅当对 $R(U)$ 的任一关系 r ，如果存在元组 (x, y_1, z_1) 和 (x, y_2, z_1) ，则必然存在元组 (x, y_2, z_1) 和 (x, y_1, z_2) (即，交换元组在属性组 Y 上的值之后，所得到的新元组必在 r 中)。

CSDN @Moehoo猛虎

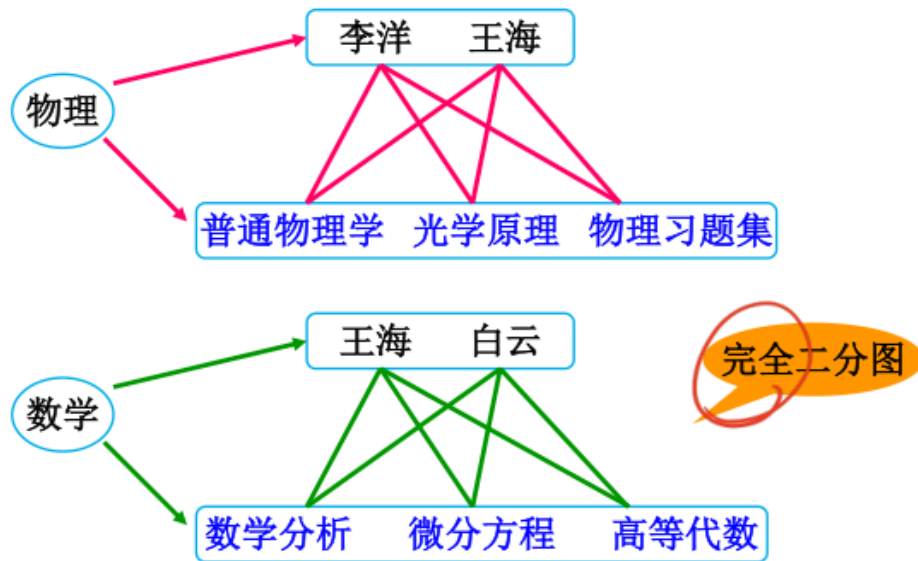
解释理解：对于同一个 x 来说，如果 z_1 有 y_1 ，同时别的 z 有 y_2 ；那么 z_1 也必定有 y_2
同理，对于同一个 x 来说，如果 z_2 有 y_2 ，同时别的 z 有 y_1 ，那么 z_2 也必定有 y_1

例如：在关系模式TEACH中有 $C \twoheadrightarrow T$

直观上看，若 $X \twoheadrightarrow Y$ ，则 X 的一个值唯一决定一组 Y 值，且这组值与 X, Y 之外的属性值无关

课程C	教员T	参考书B
物理	李洋	普通物理学
物理	李洋	光学原理
物理	李洋	物理习题集
物理	王海	普通物理学
物理	王海	光学原理
物理	王海	物理习题集
数学	王海	数学分析
数学	王海	微分方程
数学	王海	高等代数
数学	白云	数学分析
计算	白云	数学分析
.....		

由前面例子，可看出X、Y、Z之间有下列关系：



- 函数依赖可看成是多值依赖的特例：若 $X \rightarrow Y$ ，则 $X \twoheadrightarrow Y$

多值依赖与函数依赖的区别

(1) 函数依赖 $X \rightarrow Y$ 的有效性仅取决于X、Y，与X、Y之外的属性无关：

$X \rightarrow Y$ 在 $\pi_{XY}(R)$ 上成立 $\iff$ $X \rightarrow Y$ 在 $\pi_W(R)$ 上成立

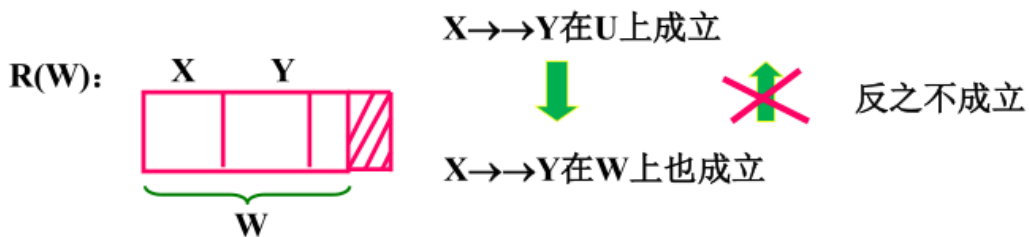
其中W满足 $XY \subseteq W \subseteq U$ (U是关系模式R的属性集)。



多值依赖 $X \twoheadrightarrow Y$ 的有效性与X、Y之外的属性范围有关：

若 $X \twoheadrightarrow Y$ 在U上成立，则在W ($XY \subseteq W \subseteq U$) 上也成立，但反之不然。

可缩小范围但不一定能扩大范围



2. 多值依赖的标准定义及实例图^Q

【定义 1.1】

设 $R(U)$ 是属性集 U 上的一个关系模式。 X, Y, Z 是 U 的子集, 并且 $Z = U - X - Y$ 。关系模式 $R(U)$ 中多值依赖 $X \twoheadrightarrow Y$ 成立, 当且仅当对 $R(U)$ 的任一关系 r , 给定的一对 (x, z) 值, 有一组 Y 的值, 这组值仅仅决定于 x 值而与 z 值无关。

例如, 在关系模式 Teaching 中, 对于一个 (物理, 光学原理) 有一组 T 值 {李勇, 王军}, 这组值仅仅决定于 C 上的值 (物理)。也就是说对于另一个 (物理, 普通物理学), 它对应的一组 T 值仍然是 {李勇, 王军}, 尽管此时参考书 B 的值已经改变了。因此 T 多值依赖于 C, 即 $C \twoheadrightarrow T$ 。

[对教材的补充说明] 对于一个 (物理, 李勇) 也是如此, 即 $C \twoheadrightarrow B$ 也成立。

4.3.2 关系模式规范化理论

1. 第一范式 (1NF)

- 不存在复合属性及多值属性
- 即: 属性不可再分

规范方法:

- 将复合属性用各子属性代替, 称为简单属性
- 将含有多值属性的表分解成两张表, 一张表由主键和简单属性构成, 另外一张表由多值属性和主键

2. 第二范式 (2NF)

- 满足 1NF
- 所有非主键属性都完全依赖于主键
(如果是复合主键 (A,B), 其余属性必须完全依赖于这两个属性, 不能只依赖 A 或只依赖 B)

规范方法:

- 将部分函数依赖的属性从表中移出, 放入一个新关系
- 将这些属性的决定因子作为主键放到新关系中

学号	姓名	班号	学院	课程号	课程名	成绩
14101	张三	99141	自动化	A101	高等数学	80
14101	张三	99141	自动化	A204	计算机网络	83
14122	李四	99141	自动化	A101	高等数学	91
14213	王五	99142	自动化	A101	高等数学	76
08113	赵六	99081	计算机	A107	大学物理	88
08218	钱七	99082	计算机	A204	计算机网络	69

其中: 学号和课程号构成主键 如果不是 2NF, 如何解决?

将上述的表分解为如下三张表

原来部分依赖于学号

学号	姓名	班号	学院
14101	张三	99141	自动化
14122	李四	99141	自动化
14213	王五	99142	自动化
08113	赵六	99081	计算机
08218	钱七	99082	计算机

原来完全依赖于学号的非主属性

学号	课程号	成绩
14101	A101	80
14101	A204	83
14122	A101	91
14213	A101	76
08113	A107	88
08218	A204	69

原来部分依赖于课程号

课程号	课程名
A101	高等数学
A204	计算机网络
A107	大学物理

可以写成如下关系模式：

- (1) 学生表 (学号, 姓名, 班号, 学院)
- (2) 课程表 (课程号, 课程名)
- (3) 选课表 (学号, 课程号, 成绩)

考试时写成这样就好不用把上面表写出来

3.第三范式 (3NF)

- 满足1NF
- 所有非主键属性均不存在对键的传递函数依赖

(非主键属性不能由其他非主键属性或其组合决定)

(例如, (ANOP) 的主键为A, N、O、P不能由单个的N、O、P或他们的组合决定)

(满足了2NF, 依赖于主键是肯定的, 但并不妨碍同时传递依赖于主键, 就像学院依赖于学号, 但学院同时也依赖于班号, 就形成了

学号→班号→学院, 的传递依赖)

规范方法:

- 先确保满足2NF
- 将传递依赖的属性及其中间属性移出构成新表
- 将不存在传递依赖的属性留下

看看以下关系表是否满足3NF?

学号	姓名	班号	学院
14101	张三	99141	自动化
14122	李四	99141	自动化
14213	王五	99142	自动化
08113	赵六	99081	计算机
08218	钱七	99082	计算机

将上述的表分解为如下两张表

学号	姓名	班号
14101	张三	99141
14122	李四	99141
14213	王五	99142
08113	赵六	99081
08218	钱七	99082

班号	学院
99141	自动化
99142	自动化
99081	计算机
99082	计算机

可以写成如下关系模式：

(1) 学生表 (学号, 姓名, 班号)

(2) 班级表 (班号, 学院)

4.BCNF范式 (改进的3NF)

- 满足1NF
- 所有的依赖关系的决定因子必须是候选键

(即不存在主属性对键的传递函数依赖或部分依赖)

主属性：构成候选键的是主属性，不构成的是非主属性

- 若R满足BCNF，则一定满足3NF，但满足3NF并不一定就满足BCNF

■ 假设一个教师只能讲一门课程，一门课有多个教师讲，一个学生可以选多门不同的课

学生	教师	课程
丁一	袁老师	数据库
马二	李老师	计算机网络
马二	袁老师	数据库
张三	王老师	计算机网络
张三	赵老师	数据库
李四	袁老师	数据库
王五	李老师	计算机网络

候选键有：(学生, 教师)

(学生, 教师) → 课程
教师 → 课程

如果不是BCNF，如何解决？

这个例子中，有部分函数依赖，不满足2NF，先拆解，拆解完，发现满足了3NF，也满足了BCNF

将上述的表分解为如下两张表

学生	教师
丁一	袁老师
马二	李老师
马二	袁老师
张三	王老师
张三	赵老师
李四	袁老师
王五	李老师

教师	课程
袁老师	数据库
李老师	计算机网络
王老师	计算机网络
赵老师	数据库

5.第四范式 (4NF)

- 满足1NF
- 任一非平凡多值依赖 $X \twoheadrightarrow Y$, X 含有候选键

看看以下关系表是否满足4NF?

仓库	保管员	货品
101仓库	王xx	洗衣粉
101仓库	王xx	香皂
101仓库	陈xx	洗衣粉
101仓库	陈xx	香皂
102仓库	刘xx	微波炉
102仓库	刘xx	电吹风
102仓库	李xx	微波炉
102仓库	李xx	电吹风
102仓库	张xx	微波炉
102仓库	张xx	电吹风

如何解决?

- 对于 $U=X+Y+Z$, 如果有 $X \twoheadrightarrow Y$, 则将 U 分解 XY 和 XZ 两张表

将上述的表分解为如下两张表

仓库	保管员
101仓库	王xx
101仓库	陈xx
102仓库	刘xx
102仓库	李xx
102仓库	张xx

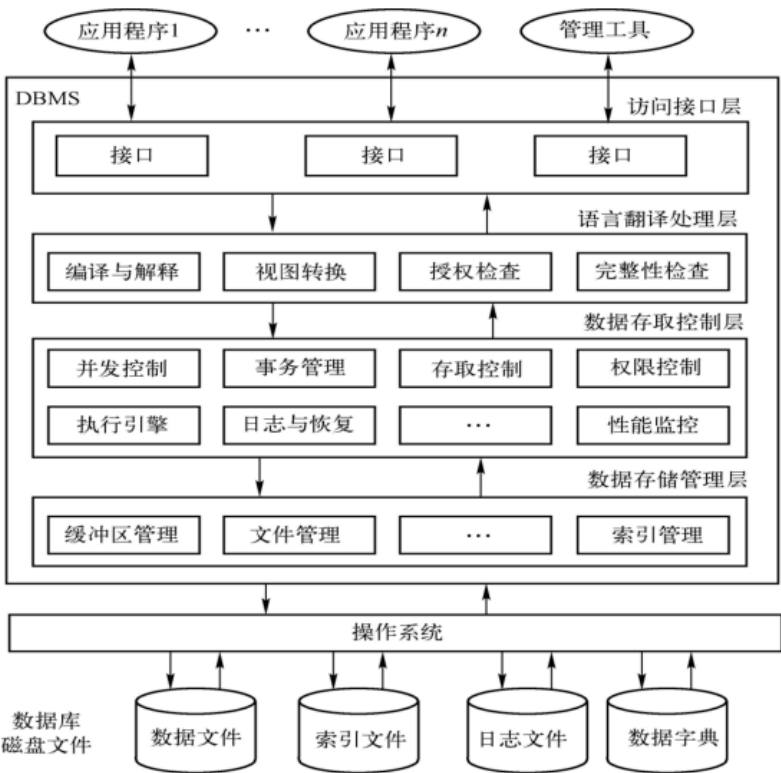
仓库	货品
101仓库	洗衣粉
101仓库	香皂
102仓库	微波炉
102仓库	电吹风

可以写成如下关系模式:

- (1) 管理表 (仓库, 保管员)
- (2) 存储表 (仓库, 货品)

第五章 数据库管理

4、数据库管理系统(DBMS)的系统结构



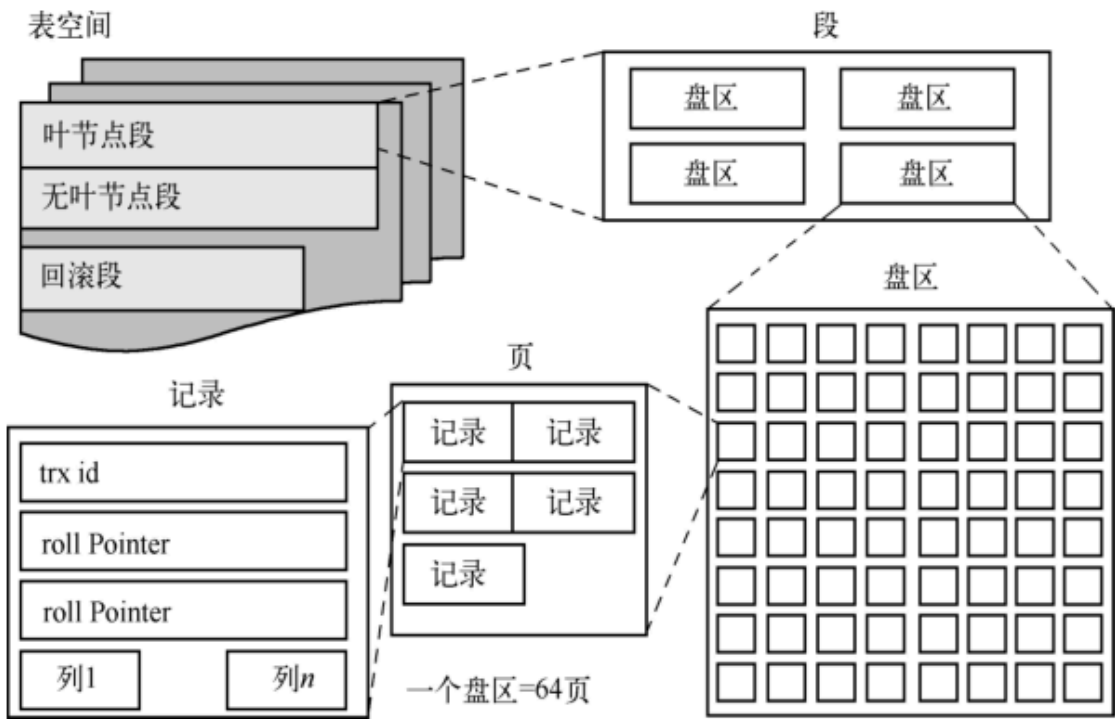
5.1数据库存储管理

5.1.1数据库存储结构

1.数据库存储结构可以分为：逻辑存储结构、物理存储结构

2.逻辑存储结构：

表空间、段、盘区、页、数据记录

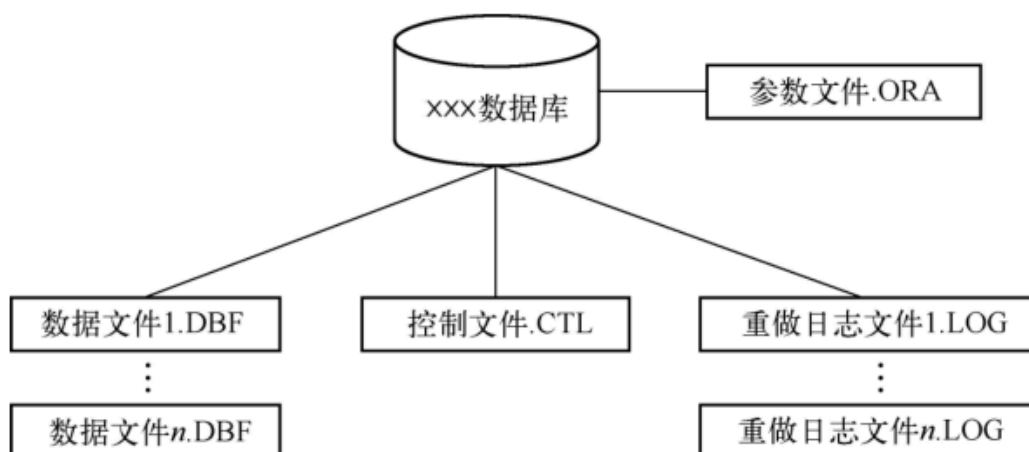


包含关系：

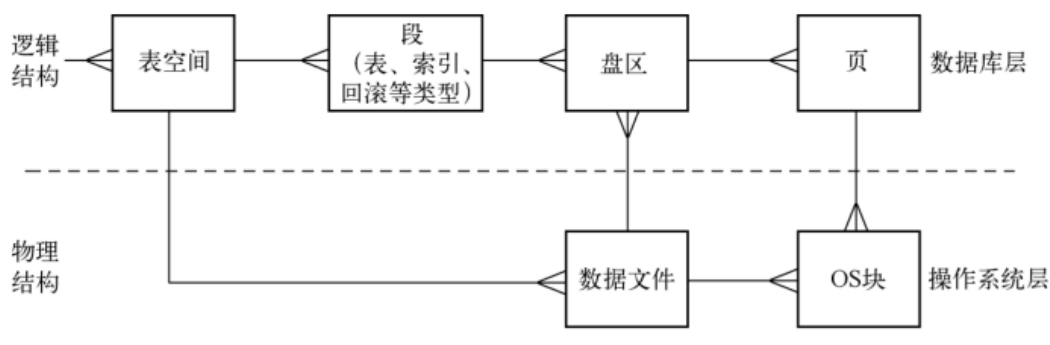
- 一个数据库逻辑上由若干个表空间组成
 - 一个表空间由若干个段组成
 - 一个段由若干个盘区组成
 - 一个盘区由若干内存页组成
 - 一个页由若干数据记录组成
-
- 段：数据段、索引段、临时段.....
 - 盘区：**内存中**连续多页的存储空间（与磁盘扇区（磁盘的概念）不同）
 - 页：数据库中**内存缓冲区**与**磁盘存储空间**交换数据的最小单位，默认页大小为8KB
 - 数据记录：即关系表中的元组

3.物理存储结构

- 每个数据库至少有：数据文件、控制文件、重做日志文件
- 重做日志文件：存储事务程序访问数据库的操作信息



4.逻辑存储结构于物理存储结构的关系



- 一个数据库表空间依托一个或多个数据文件用于存储数据
- 一个数据文件在内存中可能占用多个盘区
- 一个内存页对应操作系统磁盘数据文件的一个块或者多个块

- 文件块：记录、块头区、空闲区
块头区：链接指针、时间戳等信息
- 若当前文件块没有存储完整记录的空间，则需将该记录存储到另一空闲的文件块内

课本P161例题，再好好看看

- 删除和插入的的算法，三种

2.变长记录

- 变长记录格式：



图5-9 变长记录格式

- 分为记录固定区和记录变长区
- 变长字段的偏移量和长度信息存放在记录固定区
- 空值位图：标记记录内各字段是否为空值，“1”表示为空值
- 例子：



图5-10 Student表的变长记录存储示例

- 各个变长字段，偏移值、长度值分别占一个字节

分槽结构：

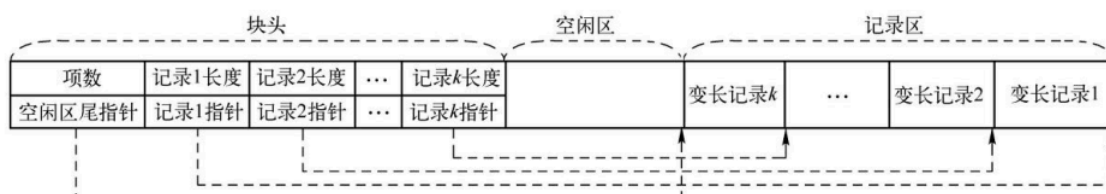


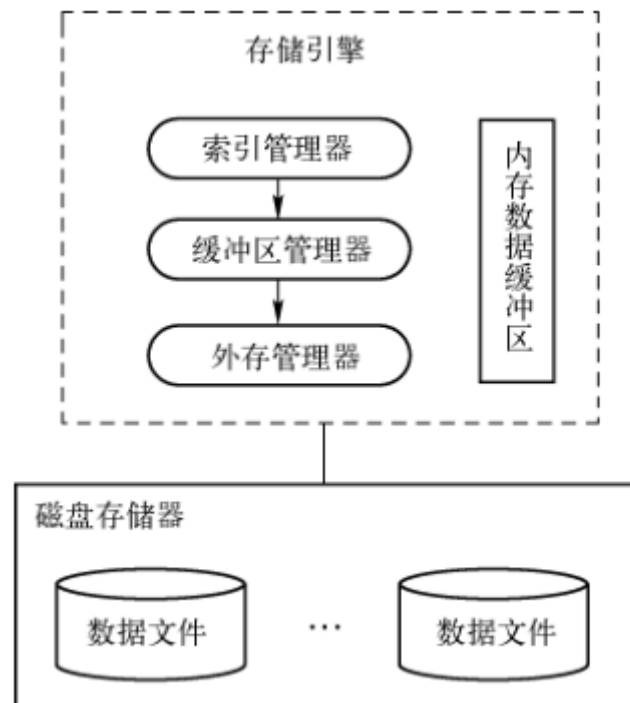
图5-11 文件块的分槽结构存储变长记录

- 项数：文件块中存储的记录数
- 块头区有项数各记录指针单元和记录长度单元
- 在文件块的记录区块尾开始依次存储各个变长记录
- 删除某记录时，在块头区移动其后的指针和长度单元，在记录区移动其后的边长记录

5.1.2.2数据文件的记录组织

- 1.顺序组织方式
- 2.索引文件组织
- 3.散列文件组织
- 4.堆文件组织

5.1.3存储引擎



- 负责数据存储、数据检索、数据更新
- 包含的模块：索引管理器、内存管理器、外存管理器、内存数据缓冲区等

5.2数据库索引

1.索引作用：

- 大大加快数据的查询速度，这也是创建索引的最主要动因
- 加快表与表之间的连接速度

5.2.1 B+树索引

- B+树的根节点和中间层子节点只保存索引的**搜索键值以及下层节点指针**
- 叶节点不但保存索引的搜索键值，还保存对应的数据记录指针和链表指针

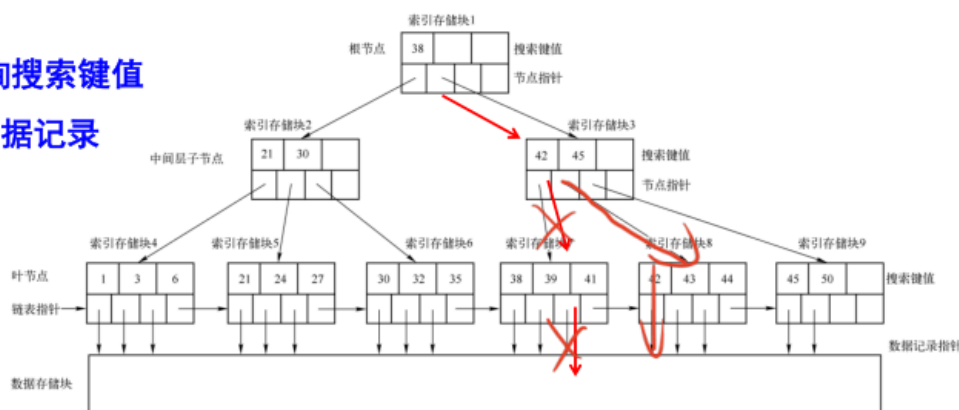
B+树结构

B+树结点的最大孩子个数称为树的阶，通常用 m 表示。一棵 m 阶B+树要么是空树，要么满足以下定义：

- ① 一个结点的结构是： $(K_1, P_1, K_2, \dots, K_n, P_n)$ ，其中 P 为指向子树的指针， K 为关键字；每个分支节点最多有 m 棵子树（孩子节点）；
- ② 所有分支节点（可视为索引的索引）中仅包含他的各个子节点（即下一级的索引块）中关键字的最大值及指向其子节点的指针。
- ③ 对于非根的所有结点，至少拥有 $\lceil m/2 \rceil$ 棵子树，即至少拥有 $\lceil m/2 \rceil$ 个关键字
- ④ 为了满足阶的定义，任何结点最多拥有 m 个子树，即最多拥有 m 个关键字
- ⑤ B+树的所有叶节点包含全部关键字及指向相应记录的指针，叶节点中将关键字按大小顺序排列，并且相邻叶节点按大小顺序相互链接起来

例5-3：查询搜索键值

等于42的数据记录



5.2.1.1 B+树记录插入

“先变胖，后变高”

- 叶节点关键字个数小于 m
- 插入后叶节点关键字个数 $> m$
- 插入的值比叶节点中最大的值还要大

B+树记录插入

- 插入的操作全部都在叶子结点上进行，且不能破坏关键字自小而大的顺序；
- 当插入关键字后结点的关键字个数大于 m ，需要进行“分裂”。
- B+树的插入有四种情况：
 - ① 若被插入关键字所在的结点，其含有关键字数目小于 m ，则直接插入；
 - ② 若被插入关键字所在的结点，其含有关键字数目等于 m ，则需要将这个结点分为左右两部分，中间的结点放到父节点中。假设其双亲结点中包含的关键字数小于 m ，则插入操作完成。
 - ③ 在第2种情况中，如果上移操作导致其双亲结点中关键字个数大于 M ，则应继续分裂其双亲结点。
 - ④ 若插入的关键字比当前结点中的最大值还大，破坏了B+树中从根结点到当前结点的所有索引值，此时需要及时修正后，再做其他操作。

5.2.1.2 B+树记录删除

“先变瘦，再变矮”

B+树记录删除

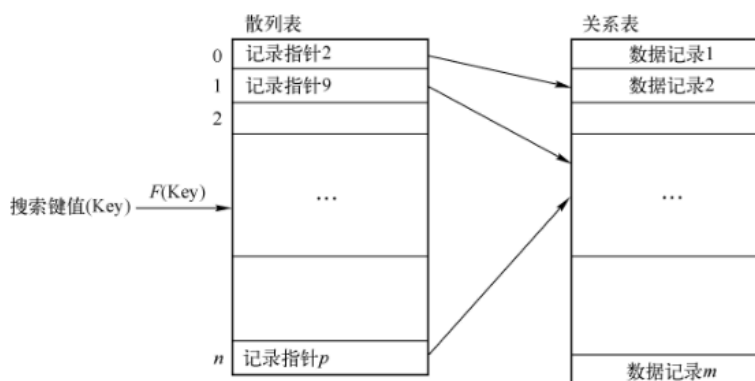
B+树的删除也和B树十分类似，它有下面几种情况：

- ① 找到存储有该关键字所在的结点时，由于该结点中关键字个数 $\geq \lceil m/2 \rceil$ ，做删除操作不会破坏 B+树，则可以直接删除；
- ② 当删除某结点中最大或者最小的关键字，就会涉及到更改其双亲结点一直到根结点中所有索引值的更改。
- ③ 当删除该关键字，导致当前结点中关键字个数小于 $\lceil m/2 \rceil$ ，若其兄弟结点中含有多余的关键字，可以从兄弟结点中借关键字完成删除操作。
- ④ 第 3 种情况中，如果其兄弟结点没有多余的关键字，则需要同其兄弟结点进行合并。
- ⑤ 当进行合并时，可能会产生因合并使其双亲结点破坏 B+树的结构，需要依照以上规律处理其双亲结点。

B+树的使用效率，还没看

5.2.2 散列索引（Hash索引）（去年本来打算考的）

- 将索引键值经过哈希函数计算后映射到一个固定大小的散列地址上，从而实现快速的数据检索
 - 存放哈希值对应记录指针的存储单元称为“桶” (Bucket)。在内存的散列索引结构中，每个桶存储记录指针或记录链表。在磁盘文件的散列索引结构中，每个桶则存储实际数据记录。



散列函数的选择

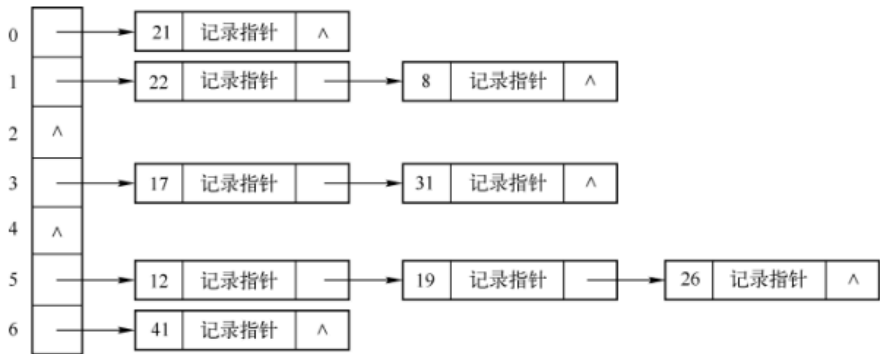
- ① 直接寻址法
- ② 数字分析法
- ③ 平方取中法
- ④ 取随机数法
- ⑤ 除留取余法

散列冲突解决

- ① 开放寻址法
- ② 再哈希法
- ③ 公共溢出区
- ④ 链地址法

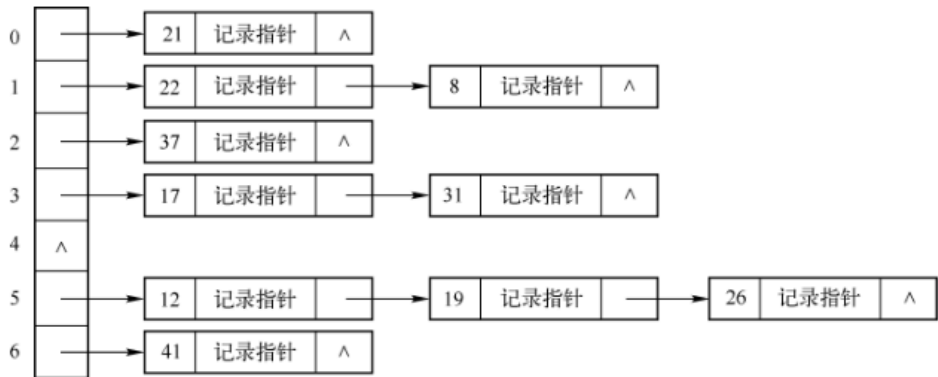
链表地址法的散列索引示例

假设散列表桶数为7，每个桶仅存储一个链表首指针，链表节点存储(键值,数据记录指针,链表指针)索引。当有多个键值映射到同一个桶时，将各键值—记录指针节点放入该桶链表中。例如，有一组键值key为{12,17,22,8,19,31,41,26,21}，其散列函数为 $F(key)=key \text{ Mod } 7$ ，使用链地址法所构建的散列表如图



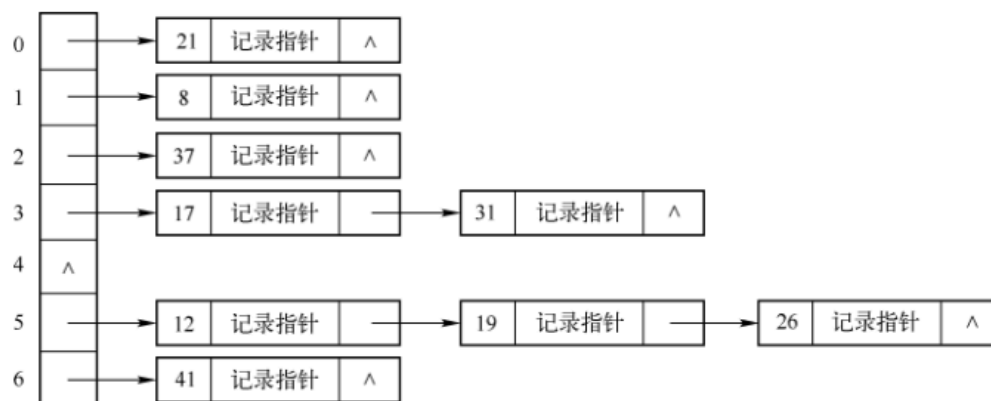
散列索引插入示例

在使用上图散列表索引的关系表中，当插入键值 $key=37$ 的数据记录时，同步计算新插入记录键值的散列值 $F(37)=2$ 。在散列表中该值对应的桶单元为空，则将新记录的索引数据 (37, 记录指针, ^) 放入一个新链表节点中，并将该节点的指针放入该桶单元; 该记录插入后，散列表索引结构如图。



散列索引删除示例

在使用上图散列索引的关系表中，当删除键值key=22的数据记录时，同步计算该删除记录的键值散列值 $F(22)=1$ 。在散列表偏移值为1的桶单元中获取链表指针，通过该指针找到含有键值22的节点，删除该节点，并将该节点的后继节点指针放入该桶单元。删除该记录后，散列表索引结构如图所示。



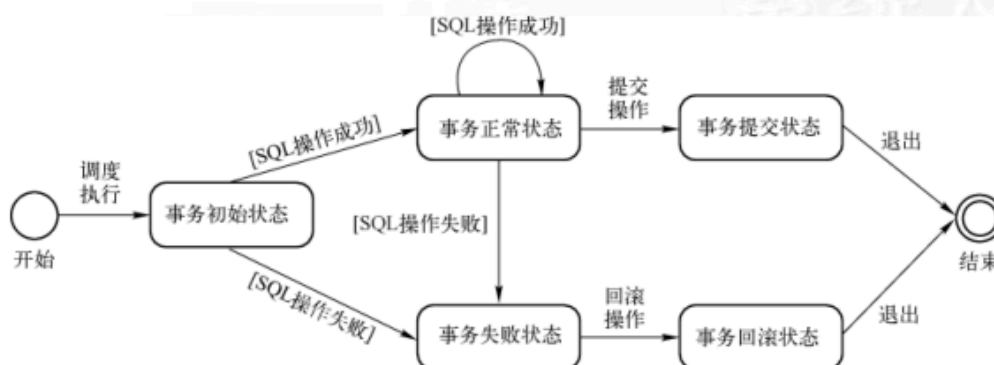
5.3 事务管理

5.3.1 事务

1. 这些操作要么都做，要么都不做，是一个不可分割的执行单位

2. 事务状态

事务状态：根据事务状态变迁，事务状态有：初始状态、正常状态、失败状态、回滚状态、事务提交、事务结束。



事务的两种结局：①提交而结束，其对DB的更新才能被其它事务访问；
②事务失败，事务所作的DB更新必须回滚。

对故障事务处理的依据：事务是已提交，还是未提交。

3. 事务的特征：

- 原子性：全做or全不做，commit or rollback
- 一致性：数据的正确性和完整性不会变

- 隔离性：多个事务并发执行时，每个事务的操作应该与其他事务的操作相互隔离，互不干扰（通过隔离级别（如**读未提交**、**读提交**、**可重复读**、**串行化**）来控制事务之间的影响，避免并发执行时出现**脏读**、**不可重复读**、**幻读**等问题）
- 持久性：一旦事务提交，其所做的修改将永久保存在数据库中，即使系统发生故障也不会丢失

5.3.2事务SQL编程

• 事务定义语句

BEGIN

SQL 语句1

SQL 语句2

...

COMMIT

BEGIN

SQL 语句1

SQL 语句2

...

ROLLBACK

- ① **BEGIN** 或 **START TRANSACTION** 标明事务的开始；
- ② **Commit** 事务执行成功，正常提交而结束
- ③ **Rollback** 事务执行失败，撤销回滚而结束

示例：

将实现转账操作的事务处理程序。假设要将 200.00 从 Alice 的账户转账到 Bob 的账户。

BEGIN; -- 开始事务

UPDATE accounts -- 从 Alice 的账户中扣除金额
SET balance = balance - 200.00
WHERE account_name = 'Alice';

UPDATE accounts -- 向 Bob 的账户中增加金额
SET balance = balance + 200.00
WHERE account_name = 'Bob';

INSERT INTO transactions (from_account, to_account, amount) -- 记录交易
VALUES (
 (SELECT account_id FROM accounts WHERE account_name = 'Alice'),
 (SELECT account_id FROM accounts WHERE account_name = 'Bob'),
 200.00
);

COMMIT; -- 提交事务

savepoint创建保存点：

在PostgreSQL中，SAVEPOINT语句用于创建一个保存点，可以在事务中设置保存点，以便在后续的操作中可以回滚到该保存点。这在处理复杂的事务逻辑时非常有用，可以将事务分成多个逻辑部分，并在需要时部分回滚。

- ① 创建保存点：SAVEPOINT sp1;
- ② 回滚到保存点：ROLLBACK TO SAVEPOINT sp1;
- ③ 释放保存点：RELEASE SAVEPOINT sp1;

下面是一个简单的示例，演示如何在PostgreSQL中使用SAVEPOINT来管理事务：

BEGIN; -- 开始事务

UPDATE accounts SET balance = balance - 100 WHERE account_id = 123; -- 扣除账户123的余额100

SAVEPOINT sp1; -- 创建保存点

UPDATE accounts SET balance = balance + 100 WHERE account_id = 456; -- 增加账户456的余额100

ROLLBACK TO SAVEPOINT sp1; -- 如果后续操作出现问题，可以回滚到保存点sp1

COMMIT; -- 提交事务

在上面的示例中，首先开始一个事务，然后扣除账户123的余额，接着创建一个保存点sp1，然后增加账户456的余额。如果后续操作出现问题，可以通过ROLLBACK TO SAVEPOINT sp1;回滚到保存点sp1，撤销增加账户456余额的操作。最后通过COMMIT;提交事务。

(这里的rollback一定会执行)

Deepseek的解释：

为什么注释有误导性？

代码中的注释提到“如果后续操作出现问题，可以回滚到保存点”，但实际没有条件判断或错误处理逻辑。这里的

ROLLBACK TO SAVEPOINT 是无条件执行的，无论后续操作是否出错都会触发。

若需根据错误触发回滚，需结合数据库的异常处理机制（如PL/pgSQL的 EXCEPTION 块）或应用程序逻辑，但原代码中未体现。

5.4并发控制

5.4.1并发执行可能会出现的问题

1.脏读：当一个事务对数据进行了修改但尚未提交时，另一个事务读取了这个未提交的数据，这就造成了脏读

(如果修改数据的事务最终回滚，那么读取到的数据就是无效的，这会对系统造成混乱和错误的结果。)

为了避免脏读，数据库系统通常会采取事务隔离级别中的“读已提交（Read Committed）”或更高级别的隔离级别，确保一个事务只能读取到已经提交的数据，避免读取到未提交的数据。通过合适的并发控制机制和隔离级别，可以有效地避免脏读问题，保证数据操作的正确性和一致性。

2.不可重复读：在一个事务内，两次读取同一数据，但得到的结果却不一样。

(可能是因为在两次读取之间，另一个事务对数据进行了修改并提交，导致第一个事务在第二次读取时得到了不同的数据。)

不可重复读可能会导致数据的不一致性，因为同一数据在不同时间点的读取结果不一致。为了避免不可重复读问题，数据库系统通常会采取事务隔离级别中的“可重复读（Repeatable Read）”或更高级别的隔离级别，确保在一个事务内多次读取同一数据时，得到的结果是一致的，不会受到其他事务的影响。
通过合适的并发控制机制和隔离级别，可以有效地避免不可重复读问题，保证数据操作的正确性和一致性。

3.幻读：在一个事务内多次查询同一范围的数据时，第二次查询发现有新的数据行被插入，导致第二次查询返回的结果集比第一次查询时更多。

幻读可能会导致数据的不一致性，因为同一范围的数据在不同时间点查询时，结果集发生了变化。为了避免幻读问题，数据库系统通常会采取事务隔离级别中的“可重复读（Repeatable Read）”或更高级别的隔离级别，确保在一个事务内多次查询同一范围的数据时，结果集是一致的，不会受到其他事务的影响。
通过合适的并发控制机制和隔离级别，可以有效地避免幻读问题，保证数据操作的正确性和一致性。

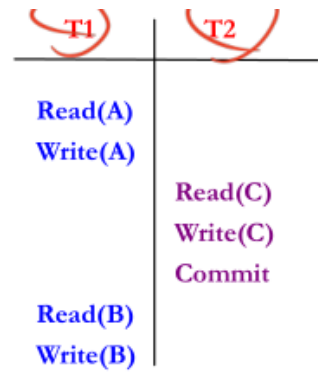
4.丢失更新：两个事务同时读取同一数据并进行修改时，后提交的事务可能会覆盖先提交的事务所做的修改，导致数据的更新操作丢失。

(例如A撤销时，把A执行过程中提交的B的数据丢失了)

丢失更新可能会导致数据的不一致性，因为后提交的事务覆盖了先提交的事务所做的修改。为了避免丢失更新问题，数据库系统通常会采取并发控制技术，如锁定机制或乐观并发控制，确保在多个事务同时修改同一数据时，能够正确处理更新操作，避免数据的丢失和覆盖。通过合适的并发控制机制，可以有效地避免丢失更新问题，保证数据操作的正确性和一致性。

5.4.2 并发事务调度

WRITE事务写操作，简记为W，
READ事务读操作，简记为R，
WT(X)：事务T写数据库元素X，
RT(X)：事务T读数据库元素X，
S表示一个调度：
 $S = R(A)W(A)R(C)W(C)R(B)W(B)$



涉及两个事务的并发调度

串行调度：

- 不同事务的活动在调度中是一个接一个执行的，没有交叉的运行。

T1	T2	T1	T2
READ(A) $A := A + A * 0.1$ WRITE(A) READ(B) $B := B + B * 0.2$ WRITE(B)	READ(A) $A := A + 10$ WRITE(A) READ(B) $B := B - 20$ WRITE(B)	READ(A) $A := A + A * 0.1$ WRITE(A) READ(B) $B := B + B * 0.2$ WRITE(B)	READ(A) $A := A + 10$ WRITE(A) READ(B) $B := B - 20$ WRITE(B)

两个串行调度的结果不同,但只要保持了数据库的一致性，最终的结果并不重要

可串行化调度：

调度等价：如有两个调度S1和S2，在DB的任意相同初始状态下，所有读出的数据都是一样的，留给DB的最终状态也是一样的，则称S1和S2是调度等价的。

可串行化调度：对一事务集，如一个并发调度与一个串行调度等价，则称此并发调度是可串行化的(Serializable)。

注意：① 对n个事务有n!种串行调度。每个串行调度执行结果可能不一样，可串行化只要求调度和其中某一个串行调度等价。
② 不同的可串行化调度是不一定等价的。

- 调度是可串行化的：多个事务交叉调度的结果与某一个串行调度的结果相同
- DBMS认为事务串行调度的结果保持了数据库的一致性，都是正确的
- 如果调度是可串行化的，系统认为其调度是一个正确的调度，保持了数据库的一致性

T1	T2		T1	T2
READ(A) $A := A + A * 0.1$ WRITE(A) READ(B) $B := B + B * 0.2$ WRITE(B)	READ(A) $A := A + 10$ WRITE(A) READ(B) $B := B - 20$ WRITE(B)	→	READ(A) $A := A + A * 0.1$ WRITE(A) READ(B) $B := B + B * 0.2$ WRITE(B)	READ(A) $A := A + 10$ WRITE(A) READ(B) $B := B - 20$ WRITE(B)

- 如果并行调度与串行调度的结果相同，因此该调度是可串行的调度

总结一下就是：串行调度都是对的，如果这个并行调度和串行调度效果相同，那么这个并行调度也是对的

不可串行化调度：

不可串行化调度

对于由多个事务组成的调度序列，如果这些事务的任何串行调度的结果都不与该调度序列的结果相等，则该调度是不可串行化的调度。

5.4.3 数据库的锁机制

- 为了实现可串行化调度，需要制定某种规则（协议），只要事务按照该协议调度执行，就可以保证调度可串行化。
- 现有数据库主要采用加锁方式来保证调度执行的可串行化

锁的基本类型：

- 排它锁（eXclusive，X锁，用于写操作）
- 某数据对象在**没有加任何锁的情况下**，一个事务可以**对其加X锁**，而其他事务就不能**对其再加任何锁**。
- 共享锁（Share，S锁，用于读操作）
- 一个事务对某数据对象**加了S锁后**，其他事务就不能**对其加X锁**，但可以加S锁。

加锁的相容表

T2 T2申请锁	T1	X锁	S锁	无
	X锁	否	否	是
	S锁	否	是	是

是：可以加锁 否：不能加锁

1.一级封锁协议

任一事务在**写某数据前**，必须对其加上X锁，该事务结束后才释放。

不采用S锁，读数据不用加锁。

2.二级封锁协议

满足一级封锁协议

且任一事务在**读取某数据前**，必须对其加上S锁，读完后就释放。

3.三级封锁协议

满足一级封锁协议

且任一事务在**读取某数据前**，必须对其加上S锁，**事务结束后就释放**。

不同级别锁协议比较

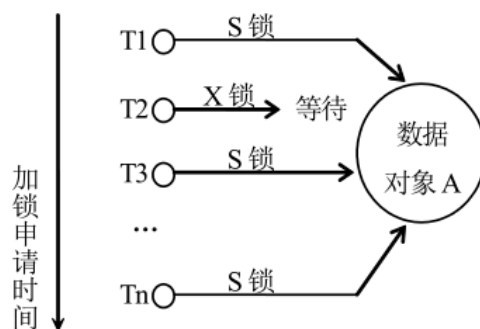
加锁协议级别	排它锁	共享锁	不丢失更新	不脏读	可重复读
一级	全程加锁	不加	是	否	否
二级	全程加锁	开始时加锁， 读完数据释放锁定	是	是	否
三级	全程加锁	全程加锁	是	是	是

4.两阶段加锁协议

- 两阶段加锁事务：在一个事务中，如果**加锁都在所有释放锁之前**，则称这样的事务加锁规则称为两阶段加锁协议(2PL协议)
- 如开始解锁，就不能再对任何对象加锁
- 如事务遵循**两阶段加锁协议**，同时遵循**先加锁后操作**原则，则该调度是**冲突可串行化**的。

5.活锁与死锁

- 活锁**：当多个事务同时申请对数据的封锁时，由于选择策略的问题导致**长时间甚至永远的等待**，称为**活锁**。



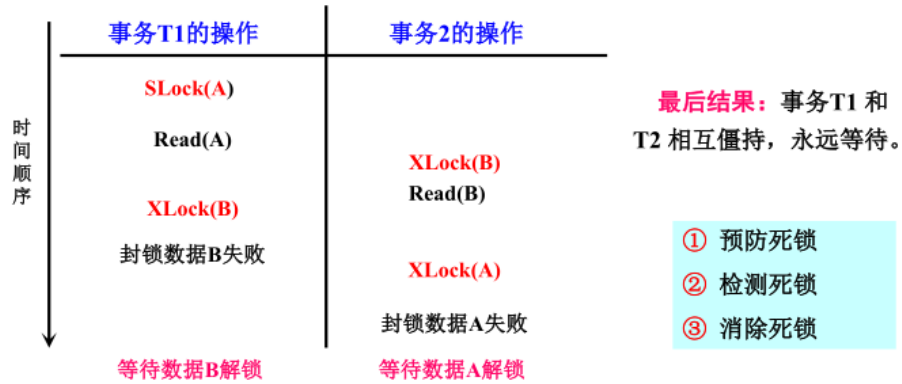
注：Ti (i=1,2,...,n) 表示第 i 个事务

- 活锁的解决方法**

采用**先来先服务**（即队列方式）并结合优先级的选择策略。

- **死锁**：在多个事务竞争共享资源的情况下，出现的相互永远等待的封锁请求，若无外力作用，这些事务永远不能向前推进，称为**死锁**。

例如：事务T1 和事务T2 并发执行，且都需要数据A 和B，假设数据A 和B 初始时为无锁状态。



5.4.4事务隔离级别

- 不同的隔离级别在事务并发执行时会采取不同的锁机制和控制策略，以确保事务操作的正确性和一致性。
- 隔离级别越高，数据库系统的并发性能通常会降低

常见的数据库隔离级别：

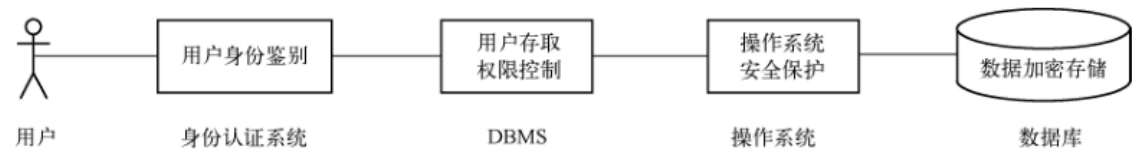
- ① **读未提交(Read Uncommitted)**：最低的隔离级别，事务可以读取其他事务未提交的数据，可能导致脏读、不可重复读和幻读问题。
- ② **读提交(Read Committed)**：事务只能读取已提交的数据，可以避免脏读问题，但仍可能出现不可重复读和幻读问题。
- ③ **可重复读(Repeatable Read)**：事务在执行过程中多次读取同一数据时，保证数据的一致性不会发生变化，可以避免不可重复读问题，但仍可能出现幻读问题。
- ④ **序列化(Serializable)**：最高的隔离级别，通过对事务进行串行化执行，确保事务之间不会相互影响，避免脏读、不可重复读和幻读问题。

事务隔离级别

隔离级别	脏读	不可重复读	丢失更新	幻读
读未提交	可能	可能	可能	可能
读已提交	不可能	可能	可能	可能
可重复读	不可能	不可能	可能	可能
可串行化	不可能	不可能	不可能	不可能

5.5 数据库安全管理

二、数据库系统安全模型



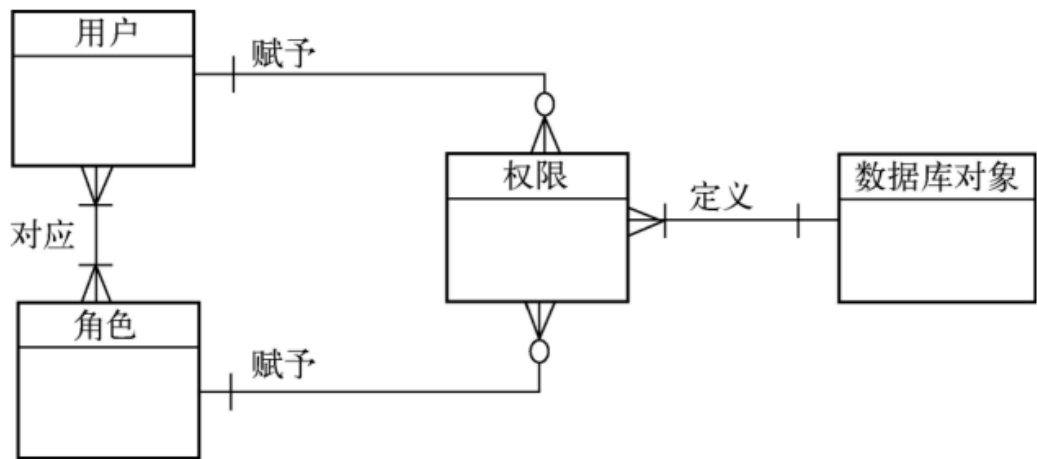
身份验证：用来确认登录用户是否是合法使用者

权限控制：通过权限机制控制用户对数据的访问

系统防护：OS系统安全机制防范非法系统访问

权限控制：通过加密算法对数据库中数据进行加密存储

三、数据库存取权限控制安全模型



给用户授权的两种方式：

- 直接给用户授权
- 利用角色给用户授权

5.5.1 用户管理

- 用户要访问数据库，必须先在DBMS中创建其账号，并成为数据库的用户。此后，用户每次访问数据库，都需要在DBMS进行身份验证，只有合法用户才能进入系统，访问操作数据库对象。
- 用户创建、用户修改、用户删除管理

1.用户创建：

1. 用户创建SQL语句

CREATE USER <用户账号名> [[WITH] option [...]];

例：创建一个新用户，其账号名字为“userA”，密码为“123456”。该用户具有登录权限（Login）和角色继承权限系统权限，但它不是超级用户，不具有创建数据库权限、创建角色权限、数据库复制权限，此外数据库连接数不受限。

```
CREATE USER "userA" WITH
  LOGIN
  NOSUPERUSER
  NOCREATEDB
  NOCREATEROLE
  INHERIT
  NOREPLICATION
  CONNECTION LIMIT -1
  PASSWORD '123456';
```

每年考 5分
字类似的这种

2.用户修改：

① 修改用户的属性

```
ALTER USER <用户名> [ [ WITH ] option [ ... ] ];
```

② 修改用户的名称

```
ALTER USER <用户名> RENAME TO <新用户名>;
```

③ 修改用户的参数值

```
ALTER USER <用户名> SET <参数项> { TO | = } { value | DEFAULT };
```

④ 重置用户参数值

```
ALTER USER <用户名> RESET <参数项>;
```

例：修改用户“userA”的账号密码为“gres123”；
同时也限制该用户的数据库连接数为10。

```
ALTER USER "userA"
  CONNECTION LIMIT 10
  PASSWORD 'gres123';
```

3.用户删除

DROP USER <用户名>;

例: 在数据库中, 删除用户 “userA”, 可以通过执行如下用户删除SQL语句实现用户删除。

DROP USER userA;

5.5.2 权限管理

- 数据库权限管理是指D B A管理员或数据库对象拥有者对其所拥有对象进行权限控制设置
- 授予权限、收回权限、拒绝权限
- 权限类别: 数据库系统权限、数据库对象访问权限、数据库对象定义权限

1.SQL语句:

GRANT <权限名> **ON** <对象名> **TO** {数据库用户名|用户角色名};

REVOKE <权限名> **ON** <对象名> **FROM** {数据库用户名|用户角色名};

DENY <权限名> **ON** <对象名> **TO** {数据库用户名|用户角色名};

↓ 被否了后, 其他的用户给他授权也不行了

例 在3.7.1节的工程项目管理系统中, DBA管理员赋予员工用户 (userA) 对部门表 (Department)、员工表 (Employee)、项目表 (Project) 和任务表 (Assignment) 的读取数据权限。

GRANT SELECT ON Department **TO** userA;

GRANT SELECT ON Employee **TO** userA;

GRANT SELECT ON Project **TO** userA;

GRANT SELECT ON Assignment **TO** userA;

5.5.3 角色管理

1. 角色管理SQL语句

① 创建角色的属性

Create ROLE <角色名> [[**WITH**] option [...]];

② 修改角色的属性

ALTER ROLE <角色名> [[**WITH**] option [...]];

③ 修改角色的名称

ALTER ROLE <角色名> **RENAME TO** <新角色名>;

④ 修改角色的参数值

ALTER ROLE <角色名> **SET** <参数项> { **TO** | **=** } { value | **DEFAULT** };

④ 删除指定角色

DROP ROLE <角色名>;

例：在工程项目管理系统中，假定需要在ProjectDB数据库内创建经理角色Role_Manager。该角色具有登录权限（Login）和角色继承权限系统权限，但它不是超级用户，不具有创建数据库权限、创建角色权限、数据库复制权限，此外数据库连接数不受限。

```
CREATE ROLE "Role_Manager" WITH
LOGIN
NOSUPERUSER
NOCREATEDB
NOCREATEROLE
INHERIT
NOREPLICATION
CONNECTION LIMIT -1;
```

注意区分“user”和“role”

例：在创建好经理角色Role_Manager后，还需要赋予该角色对数据库表Department、Employee、Project、Assignment的插入、修改、删除、查询权限。

```
GRANT SELECT,INSERT,UPDATE,DELETE ON Department TO "Role_Manager";
GRANT SELECT,INSERT,UPDATE,DELETE ON Employee TO "Role_Manager";
GRANT SELECT,INSERT,UPDATE,DELETE ON Project TO "Role_Manager";
GRANT SELECT,INSERT,UPDATE,DELETE ON Assignment TO "Role_Manager";
```

5.6 备份与恢复

5.6.1 数据库备份

数据库备份常用方法：

- 全量备份：通常定期进行
- 增量备份：在全量备份的基础上，只备份自上次备份以来发生变化的数据和日志
- 差异备份：在全量备份的基础上，备份自上次全量备份以来发生变化的数据
- 事务日志备份：只备份自上一次日志备份至今的事务日志数据的方式
- 热备份：在数据库运行时进行备份操作，不会中断数据库的正常运行
- 冷备份：在数据库停止运行时进行备份操作，会中断数据库的正常服务
- 文件备份：将数据库系统中的**系统文件、数据文件和日志文件**复制到另一个位置或存储介质

数据库备份设备：

- 磁带备份设备
- 磁盘阵列备份设备
- 网络存储设备
- 云存储服务
- 虚拟磁带库

5.6.2 数据库恢复

- 数据库恢复是指在数据库系统发生故障、数据丢失或损坏时，通过备份数据或日志记录将数据库恢复到正常运行状态的过程。

数据库恢复场景：

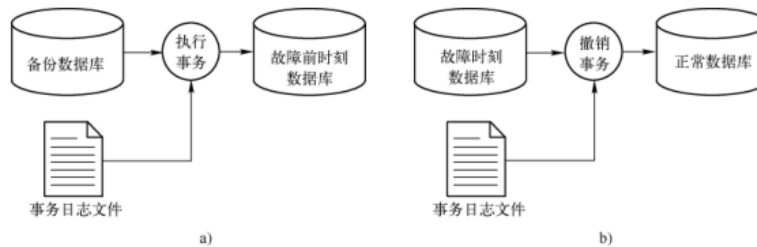
- 事务故障的数据恢复，包括以下几个方面：
 - 事务回滚
 - 事务重做
 - 事务日志恢复
 - 检查点恢复
- 系统崩溃的数据恢复，包括以下几个步骤：
 - 系统恢复启动
 - 日志重做
 - 事务回滚
 - 检查点恢复
 - 数据一致性检查
- 存储介质损坏的数据恢复
 - 存储介质评估
 - 数据恢复工具
 - 数据镜像

- 数据恢复操作
- 数据验证

数据库恢复技术：

2、数据库恢复技术

数据库恢复技术是**利用数据库备份文件和数据库事务日志文件**来实现数据库恢复处理的；根据用户恢复要求，一般采用**前滚（redo重做）事务方式**或**回滚（rollback）事务方式**恢复数据库。

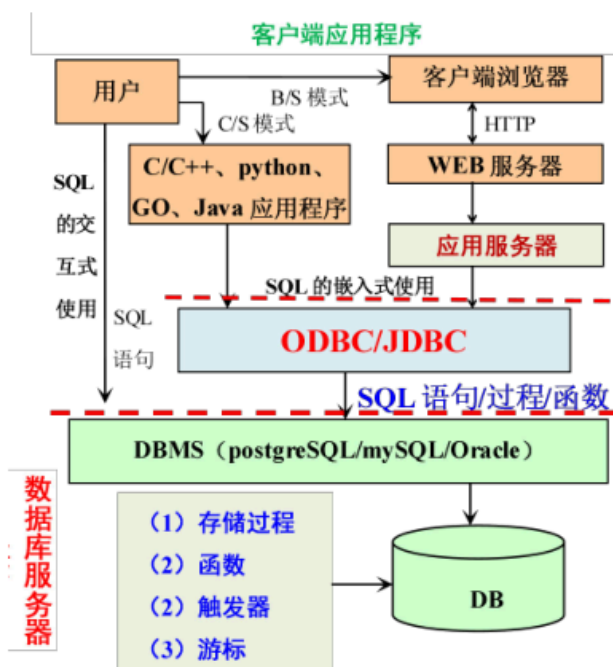


a) 前滚(redo)事务恢复方式

b) 回滚事务恢复方式

第六章 数据库编程

6.1 数据库应用程序框架



(1) 客户端应用程序

(2) 数据库服务器端程序

(3) 编程接口ODBC/JDBC

6.2 PL/pgSQL编程语言

- Postgresql数据库系统中的一种过程化的编程语言
- 主要用于创建存储过程、函数、触发器等

1.声明块

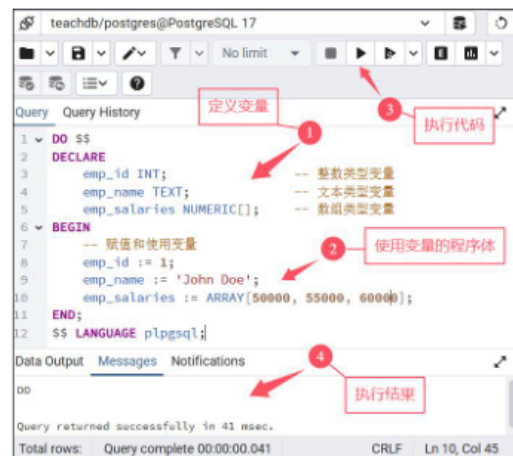
- PL/pgSQL的基本结构通常由一个声明块和一个执行块组成
- 可以通过DO语句或创建函数和存储过程来使用 PL/pgSQL。
- 整体的程序结构：

```
DO $$  
DECLARE  
variable_name data_type; -- 声明变量  
constant_name CONSTANT data_type := value; -- 声明常量  
BEGIN  
-- 执行代码  
END;  
$$ LANGUAGE plpgsql;
```

2.变量声明

- 在 PL/pgSQL 中，可以声明多种类型的变量，包括标量类型、复合类型和数组类型。

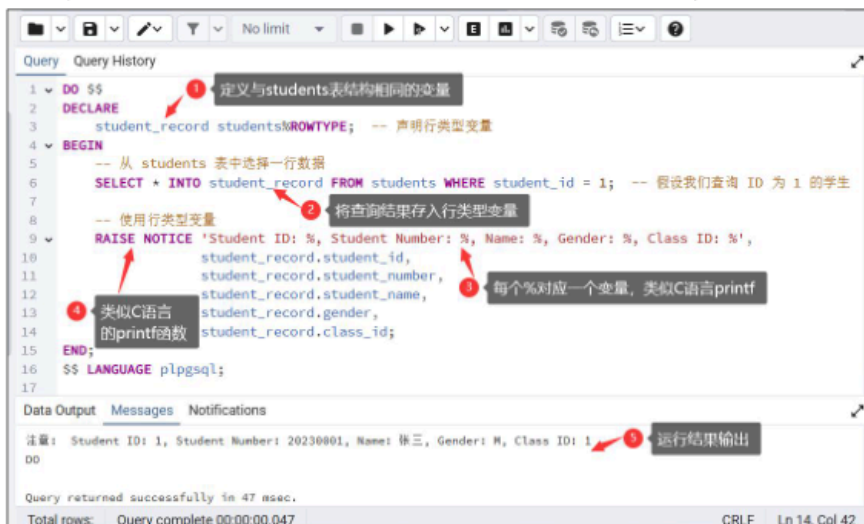
```
DO $$  
DECLARE  
    emp_id INT; -- 整数类型变量  
    emp_name TEXT; -- 文本类型变量  
    emp_salaries NUMERIC[]; -- 数组类型变量  
BEGIN  
-- 赋值和使用变量  
    emp_id := 1;  
    emp_name := 'John Doe';  
    emp_salaries := ARRAY[50000, 55000, 60000];  
END;  
$$ LANGUAGE plpgsql;
```



注意赋值使用的是":="

3.特殊类型的声明（行类型变量）

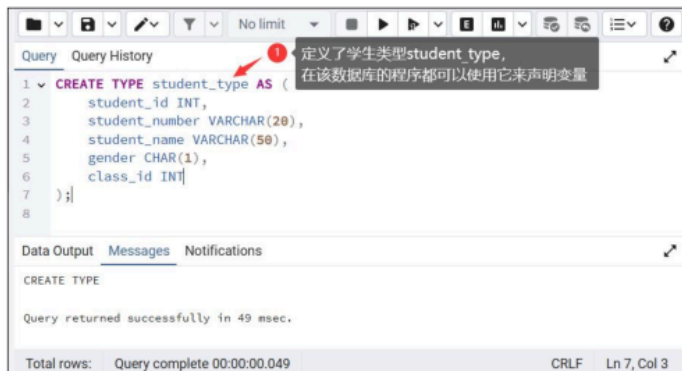
声明与指定表的行相同类型的变量，称为行类型变量。常用于数据库查询的结果赋值给变量，定义语法格式为：**变量名 表名%ROWTYPE**;



4. 复合类型的声明

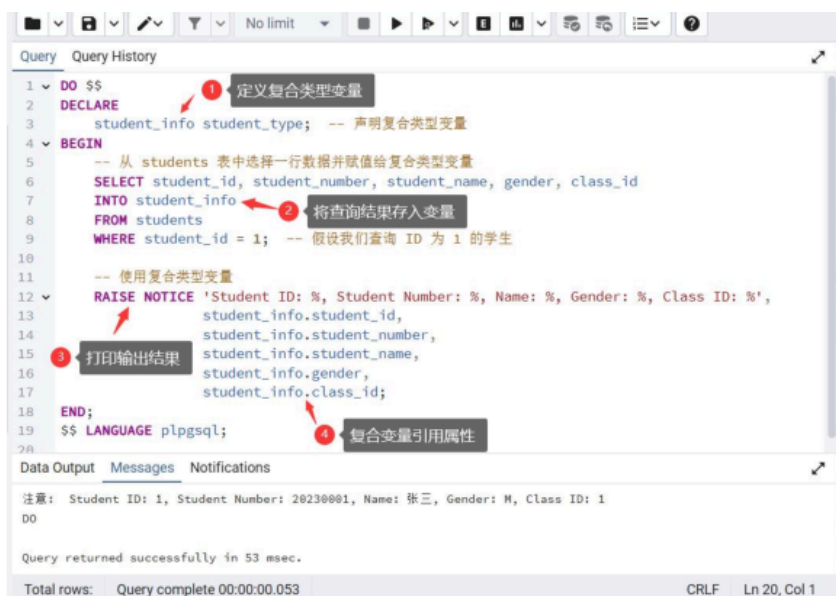
- 上述事例可以定义一个复合类型来表示学生信息，类似C语言的结构体类型

```
CREATE TYPE student_type AS (  
    student_id INT,  
    student_number VARCHAR(20),  
    student_name VARCHAR(50),  
    gender CHAR(1),  
    class_id INT  
);
```



5. 复合类型的使用

- 下面给出使用复合类型来声明变量的示例型

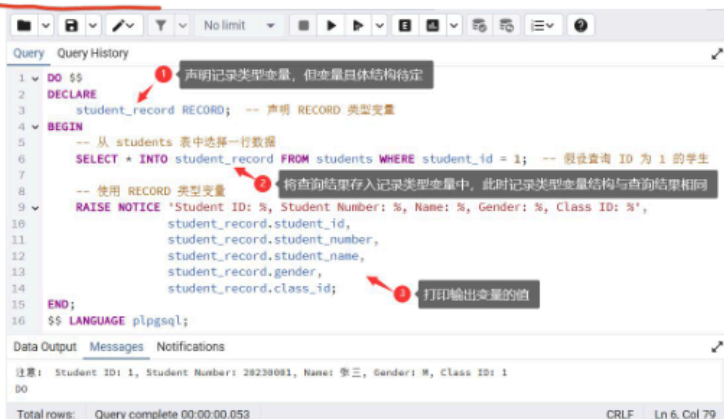


注意select语句中的into

6. 记录类型record

在 PostgreSQL 中，RECORD 类型是一种特殊的行类型，允许你在 PL/pgSQL 中存储查询结果的任意行。RECORD 类型的变量可以动态地接收查询结果的结构，这使得它非常灵活，适用于处理不确定列数或列类型的查询结果。

语法格式：变量名 RECORD;



7.record处理查询多条结果

- 使用 RECORD 类型的另一个常见用法是遍历查询结果集。以下示例展示了如何使用 RECORD 类型遍历 students 表中的所有学生。

```
1 DO $$
2 DECLARE
3     student_record RECORD; -- 声明 RECORD 类型变量
4 BEGIN
5     FOR student_record IN SELECT * FROM students LOOP -- 遍历查询结果
6         RAISE NOTICE 'Student ID: %, Student Number: %, Name: %, Gender: %, Class ID: %',
7             student_record.student_id,
8             student_record.student_number,
9             student_record.student_name,
10            student_record.gender,
11            student_record.class_id;
12     END LOOP;
13 END;
14 $$ LANGUAGE plpgsql;
```

注意: Student ID: 1, Student Number: 20230001, Name: 张三, Gender: M, Class ID: 1
注意: Student ID: 2, Student Number: 20230002, Name: 李四, Gender: F, Class ID: 1
注意: Student ID: 3, Student Number: 20230003, Name: 王五, Gender: M, Class ID: 1
注意: Student ID: 4, Student Number: 20230004, Name: 赵六, Gender: F, Class ID: 2
注意: Student ID: 5, Student Number: 20230005, Name: 钱七, Gender: M, Class ID: 2
注意: Student ID: 6, Student Number: 20230006, Name: 孙八, Gender: F, Class ID: 1
注意: Student ID: 7, Student Number: 20230007, Name: 周九, Gender: M, Class ID: 1
注意: Student ID: 8, Student Number: 20230008, Name: 吴十, Gender: F, Class ID: 2
注意: Student ID: 9, Student Number: 20230009, Name: 郑十一, Gender: M, Class ID: 2
Total rows: Query complete 00:00:00.049 CRLF Ln 14, Col 21

注意并不是record中存储了多个记录

8.if语句

基本语法:

IF condition THEN

-- 当 condition 为 TRUE 时执行的代码

ELSIF another_condition THEN

-- 当 another_condition 为 TRUE 时执行的代码

ELSE

-- 当以上条件都不满足时执行的代码

END IF;

示例:

以下是一个 PL/pgSQL 示例, 演示如何使用 IF 语句来检查学生的性别, 并根据性别输出不同的消息。

```
1 DO $$
2 DECLARE
3     student_record RECORD; -- 声明 RECORD 类型变量
4 BEGIN
5     -- 从 students 表中选择一行数据
6     SELECT * INTO student_record FROM students WHERE student_id = 1; -- 假设查询 ID 为 1 的学生
7
8     -- 使用 IF 语句检查学生的性别
9     IF student_record.gender = 'M' THEN
10         RAISE NOTICE 'Student % is male.', student_record.student_name;
11     ELSIF student_record.gender = 'F' THEN
12         RAISE NOTICE 'Student % is female.', student_record.student_name;
13     ELSE
14         RAISE NOTICE 'Student % has an unknown gender.', student_record.student_name;
15     END IF;
16 END;
17 $$ LANGUAGE plpgsql;
```

注意: Student 张三 is male.
DO
Query returned successfully in 441 msec.

9.case语句

以下是一个 PL/pgSQL 示例，演示如何使用 CASE 语句根据学生的性别输出不同的消息。

```
Query Query History
1 DO $$
2 DECLARE
3     student_record RECORD; -- 声明 RECORD 类型变量
4 BEGIN
5     -- 使用 CASE 语句在 SELECT 查询中处理性别
6     FOR student_record IN
7         SELECT student_id, student_name,
8             CASE
9                 WHEN gender = 'M' THEN 'Male'
10                WHEN gender = 'F' THEN 'Female'
11                ELSE 'Unknown'
12            END AS gender_description
13         FROM students LOOP
14         RAISE NOTICE 'Student ID: %, Name: %, Gender: %',
15             student_record.student_id,
16             student_record.student_name,
17             student_record.gender_description;
18     END LOOP;
19 END;
20 $$ LANGUAGE plpgsql;
```

Data Output Messages Notifications

注意: Student ID: 1, Name: 张三, Gender: Male
注意: Student ID: 2, Name: 李四, Gender: Female
注意: Student ID: 3, Name: 王五, Gender: Male
注意: Student ID: 4, Name: 赵六, Gender: Female
注意: Student ID: 5, Name: 钱七, Gender: Male

10.loop语句

在 PostgreSQL 的 PL/pgSQL 中，LOOP 语句用于创建一个循环，可以重复执行一段代码，直到满足特定的退出条件。LOOP 语句非常灵活，适用于需要重复执行的场景。

LOOP

-- 循环体

EXIT WHEN condition; -- 退出条件

END LOOP;

```
Query Query History
1 DO $$
2 DECLARE
3     counter INT := 0; -- 声明计数器
4 BEGIN
5     LOOP
6         counter := counter + 1; -- 增加计数
7         RAISE NOTICE 'Counter: %', counter; -- 输出计数
8         EXIT WHEN counter >= 5; -- 当计数达到 5 时退出循环
9     END LOOP;
10 END;
11 $$ LANGUAGE plpgsql;
```

Data Output Messages Notifications

注意: Counter: 1
注意: Counter: 2
注意: Counter: 3
注意: Counter: 4
注意: Counter: 5
nn
Total rows: Query complete 00:00:00.043 CRLF Ln 12, Col 1

11.while语句

在 PostgreSQL 的 PL/pgSQL 中，WHILE 语句用于创建一个基于条件的循环。与 LOOP 语句不同，WHILE 循环会在每次迭代之前检查条件，如果条件为 TRUE，则执行循环体；如果条件为 FALSE，则退出循环。

WHILE condition LOOP

-- 循环体

END LOOP;

(12) WHILE语句示例

以下是一个 PL/pgSQL 示例，演示如何使用 WHILE 语句来计算学生的总数，并输出每个学生的信息。

```
Query Query History
1 DO $$
2 DECLARE
3     student_count INT := 0; -- 学生计数器
4     student_record RECORD; -- RECORD 类型变量
5 BEGIN
6     -- 计算学生总数
7     SELECT COUNT(*) INTO student_count FROM students;
8     -- 初始化计数器
9     DECLARE
10         current_index INT := 1; -- 当前索引
11     BEGIN
12         WHILE current_index <= student_count LOOP
13             -- 从 students 表中选择一行数据
14             SELECT * INTO student_record FROM students LIMIT 1 OFFSET current_index - 1;
15
16             -- 输出学生信息
17             RAISE NOTICE 'Student ID: %, Student Number: %, Name: %, Gender: %, Class ID: %',
18                 student_record.student_id,
19                 student_record.student_number,
20                 student_record.student_name,
21                 student_record.gender,
22                 student_record.class_id;
23
24             -- 更新索引
25             current_index := current_index + 1; -- 增加索引
26         END LOOP;
27     END;
28 $$ LANGUAGE plpgsql;
```

Data Output Messages Notifications

注意: Student ID: 1, Student Number: 20230001, Name: 张三, Gender: M, Class ID: 1
注意: Student ID: 2, Student Number: 20230002, Name: 李四, Gender: F, Class ID: 1
注意: Student ID: 3, Student Number: 20230003, Name: 王五, Gender: M, Class ID: 1
注意: Student ID: 4, Student Number: 20230004, Name: 赵六, Gender: F, Class ID: 2
注意: Student ID: 5, Student Number: 20230005, Name: 钱七, Gender: M, Class ID: 2

注意：从这里可以看出，begin里面可以套begin

12.for语句

在 PostgreSQL 的 PL/pgSQL 中，FOR 语句用于创建一个循环，可以遍历一个范围的值或一个查询结果集。FOR 语句有两种主要形式：数值循环和游标循环。

FOR counter IN [REVERSE] start..end LOOP

-- 循环体

END LOOP;

- ① counter 是循环变量。
- ② start 和 end 定义了循环的范围。
- ③ REVERSE 关键字可选，用于反向循环。

```
Query Query History
1 DO $$
2 DECLARE
3     counter INT; -- 声明计数器
4 BEGIN
5     FOR counter IN 1..5 LOOP
6         RAISE NOTICE 'Counter: %', counter; -- 输出计数
7     END LOOP;
8 END;
9 $$ LANGUAGE plpgsql;
```

Data Output Messages Notifications

注意: Counter: 1
注意: Counter: 2
注意: Counter: 3
注意: Counter: 4
注意: Counter: 5

Total rows: Query complete 00:00:00.173 CRLF Ln 9, Col 21

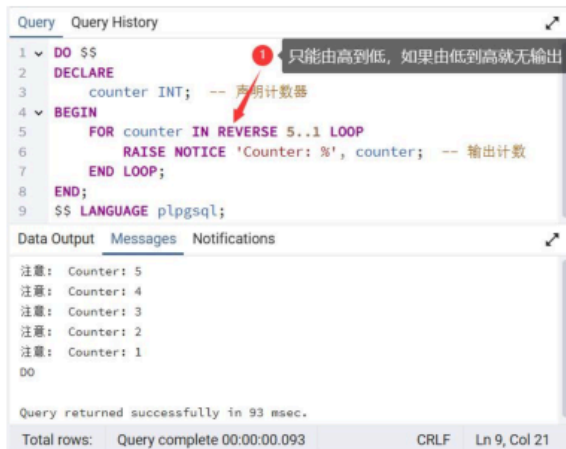
以下示例展示了如何使用游标循环遍历 students 表中的所有学生，并输出学生信息。

```
Query Query History
1 DO $$
2 DECLARE
3     student_record RECORD; -- 声明 RECORD 类型变量
4 BEGIN
5     FOR student_record IN SELECT * FROM students LOOP -- 遍历查询结果
6         RAISE NOTICE 'Student ID: %, Student Number: %, Name: %, Gender: %, Class ID: %',
7             student_record.student_id,
8             student_record.student_number,
9             student_record.student_name,
10            student_record.gender,
11            student_record.class_id;
12     END LOOP;
13 END;
14 $$ LANGUAGE plpgsql;
```

Data Output Messages Notifications

注意: Student ID: 1, Student Number: 20230001, Name: 张三, Gender: M, Class ID: 1
注意: Student ID: 2, Student Number: 20230002, Name: 李四, Gender: F, Class ID: 1
注意: Student ID: 3, Student Number: 20230003, Name: 王五, Gender: M, Class ID: 1
注意: Student ID: 4, Student Number: 20230004, Name: 赵六, Gender: F, Class ID: 2
注意: Student ID: 5, Student Number: 20230005, Name: 钱七, Gender: M, Class ID: 2
注意: Student ID: 6, Student Number: 20230006, Name: 孙八, Gender: F, Class ID: 1
注意: Student ID: 7, Student Number: 20230007, Name: 周九, Gender: M, Class ID: 1
注意: Student ID: 8, Student Number: 20230008, Name: 吴十, Gender: F, Class ID: 2
注意: Student ID: 9, Student Number: 20230009, Name: 郑十一, Gender: M, Class ID: 2

如果需要反向遍历一个范围，可以使用 REVERSE 关键字。例如，输出从 5 到 1 的数字：



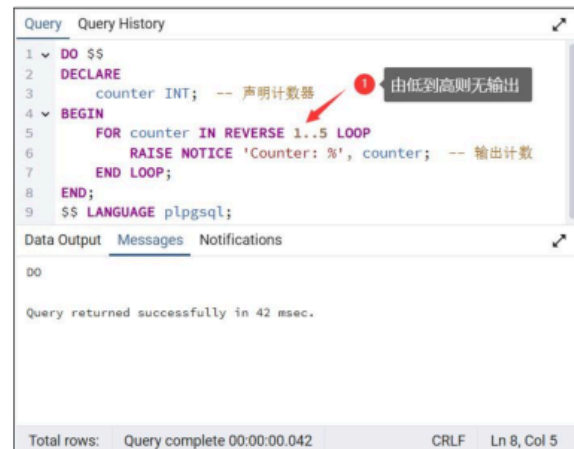
```
1 DO $$
2 DECLARE
3   counter INT; -- 声明计数器
4 BEGIN
5   FOR counter IN REVERSE 5..1 LOOP
6     RAISE NOTICE 'Counter: %', counter; -- 输出计数
7   END LOOP;
8 END;
9 $$ LANGUAGE plpgsql;
```

Data Output Messages Notifications

注意: Counter: 5
注意: Counter: 4
注意: Counter: 3
注意: Counter: 2
注意: Counter: 1
DO

Query returned successfully in 93 msec.

Total rows: Query complete 00:00:00.093 CRLF Ln 9, Col 21



```
1 DO $$
2 DECLARE
3   counter INT; -- 声明计数器
4 BEGIN
5   FOR counter IN REVERSE 1..5 LOOP
6     RAISE NOTICE 'Counter: %', counter; -- 输出计数
7   END LOOP;
8 END;
9 $$ LANGUAGE plpgsql;
```

Data Output Messages Notifications

DO

Query returned successfully in 42 msec.

Total rows: Query complete 00:00:00.042 CRLF Ln 8, Col 5

13.异常处理

在 PostgreSQL 的 PL/pgSQL 中，异常处理是指在执行过程中捕获和处理错误的机制。通过异常处理，开发者可以控制程序在遇到错误时的行为，从而提高程序的健壮性和可维护性。语法结构如下：

```
BEGIN
-- 可能引发异常的代码
EXCEPTION
WHEN exception_condition1 THEN
-- 处理异常的代码
WHEN exception_condition2 THEN
-- 处理其他异常的代码
WHEN OTHERS THEN
-- 处理所有其他未捕获的异常
END;
```

常见的异常条件

- ① **division_by_zero**：除以零的错误。
- ② **unique_violation**：唯一约束冲突。
- ③ **foreign_key_violation**：外键约束冲突。
- ④ **not_null_violation**：非空约束冲突。
- ⑤ **others**：捕获所有其他未处理的异常。

异常处理示例

在这个示例中，第一次插入成功，第二次插入会引发唯一约束冲突。异常处理块捕获到这个异常，并输出相应的消息。



```
Query Query History
1 DO $$
2 BEGIN
3   -- 尝试插入一条记录
4   INSERT INTO students (student_number, student_name) VALUES ('S001', 'John Doe');
5
6   -- 再次尝试插入相同的记录，故意引发唯一约束异常
7   INSERT INTO students (student_number, student_name) VALUES ('S001', 'Jane Doe');
8 EXCEPTION
9   WHEN unique_violation THEN
10    RAISE NOTICE 'Caught a unique violation error: Student number already exists.';
11   WHEN OTHERS THEN
12    RAISE NOTICE 'Caught an unexpected error: %', SQLERRM;
13 END;
14 $$ LANGUAGE plpgsql;
```

Data Output Messages Notifications

注意: Caught a unique violation error: Student number already exists.
DO

Query returned successfully in 57 msec.

Total rows: Query complete 00:00:00.057 CRLF Ln 14, Col 21

抛出异常处理信息

6.3 数据库函数编程

- 与C语言的函数一样，具有函数名、输入参数和输出参数
- 用户自定义函数可以在SQL查询中像内置函数一样使用

6.3.1 创建数据库函数的语法

CREATE [OR REPLACE] FUNCTION name

([[argmode] [argname] argtype [{ DEFAULT | = } default_expr] [, ...]])

[RETURNS retype | RETURNS TABLE (column_name column_type [, ...])]

AS \$\$ //\$\$用于声明函数的实际代码的开始

DECLARE

 -- 声明段

BEGIN

 --函数体语句

END;

\$\$ LANGUAGE lang_name; //\$\$ 表明代码的结束, LANGUAGE后面指明所用的编程语言

- (1) **name**: 要创建的函数名;
- (2) **OR REPLACE**: 覆盖同名的函数;
- (3) **argmode**: 函数参数的模式可以为IN、OUT或INOUT，缺省值是IN。
- (4) **argname**: 形式参数的名字。
- (5) **RETURNS**: 返回值; **RETURNS TABLE**: 返回二维表

6.3.2 示例

创建名为countRecords()的函数统计STUDENT表的记录数。

```
CREATE OR REPLACE FUNCTION countRecords ()  
RETURNS integer AS $count$  
declare  
    count integer;  
BEGIN  
    SELECT count(*) into count FROM STUDENT;  
    RETURN count;  
END;  
$count$ LANGUAGE plpgsql;
```

- \$count\$和\$\$的作用是一样的，可以把里面的count去掉，也是一样的
- 注意returns后面跟的是返回数据的类型，而不是变量名
- return后面跟的是返回的变量名

(4) 自定义带参数的函数示例

传递给函数参数都可以用\$1、\$2的标识符来表示。为了增加可读性，可以为参数声明别名，别名和数字标识符均可指向该参数值。

定义输入参数为“销售价格”的函数，返回折扣价格。

```
CREATE FUNCTION discount(salePrice real) RETURNS real AS $$  
BEGIN  
    RETURN salePrice * 0.85;  
END;  
$$ LANGUAGE plpgsql;
```

6.3.3 数据库函数执行、调用

执行：

如果程序员需要在查询窗口执行函数，语法形式如下：

select 函数名 (参数) ;

或者： **select * from** 函数名 (参数) ;

例如： **select** countRecords ();

或者： **select * from** countRecords ();

调用：

如果需要在函数或存储过程中调用数据库函数，语法形式如下：

select into 自定义变量 from 函数名 (参数) ;

CREATE OR REPLACE FUNCTION testExec()

returns integer AS \$\$

declare

rec integer;

BEGIN

select into rec countRecords();

//如果不关心countRecords()的返回值，则可用 **PERFORM countRecords()** 代替

return rec;

END;

\$\$ LANGUAGE plpgsql;

2025.4.10

注意，这里调用countRecord()函数有语法错误，

可以依据上面的来改正，也可以用下面的：

```
-- 正确顺序: SELECT countRecords() INTO rec
SELECT countRecords() INTO rec;
```

6.4数据库游标

- 游标 (Cursor) 是一种临时的数据库对象
- 用来存放从数据库表中查询返回的数据记录
- 提供了从结果集中提取并分别处理每一条记录的机制
- 游标总是与一条SQL查询语句相关联
- 游标包括：SQL语言的查询结果，指向特定记录的指针

6.4.1使用步骤

1.声明游标

- 有两种方法，一种用于声明非绑定的游标变量一种用于声明绑定的游标变量

(1)

(1) 在存储过程中游标类型的变量。例如：游标变量 refcursor;

refcursor是关键字；

此时，游标变量还没有绑定查询语句，因此不能访问游标变量。

refcursor关键字专门用于声明非绑定的游标变量

(2)

(2) 使用游标专有的声明语法，如：

游标名 CURSOR [(arguments)] FOR query;

其中arguments为由逗号分隔的参数列表，用于打开游标时向游标传递参数，类似于存储过程或函数的形式参数；query是select数据查询语句，返回的值存储在游标变量中。

这种声明方式是用于在声明时绑定查询语句

示例：

(1) **curStudent CURSOR FOR SELECT * FROM student;**

(2) **curStudentOne CURSOR (key integer) IS**

SELECT * FROM student WHERE SID = key;

- 声明完之后，游标里面还没有可以访问的数据

2.打开游标

- 有三种open语句，其中两种用于未绑定的游标变量，另外一种用于已绑定的游标变量

(1) & (2) 用于未绑定的游标：

(1) **OPEN FOR:**

其声明形式为：

OPEN unbound_cursor FOR query;

打开未绑定的游标变量，其query查询语句是返回记录的SELECT语句。

例如：**OPEN curVars1 FOR SELECT * FROM student WHERE SID = mykey;**

(2) **OPEN FOR EXECUTE**

其声明形式为：**OPEN unbound_cursor FOR EXECUTE query-string;**

打开未绑定的游标变量。**EXECUTE将动态执行查询字符串。**例如：

OPEN curVars1 FOR EXECUTE 'SELECT * FROM ' || quote_ident(\$1);

注意：\$1是指由存储过程传递的第1个参数。

(3) 用于打开绑定的游标

(3) 打开一个绑定的游标，其声明形式为：

OPEN bound_cursor [(argument_values)];

仅适用于绑定的游标变量，只有当该变量在声明时包含接收参数，才能以传递参数的形式打开该游标，参数将传入到游标声明的查询语句中，例如：

OPEN curStudent;

OPEN curStudentOne ('20160230302001');

3.从游标中获取数据

其声明形式为：**FETCH cursor INTO target;**

FETCH命令从游标中读取下一行记录的数据到目标中，读取成功与否，可通过PL/SQL内置系统变量**FOUND**来判断。

例如：

FETCH curVars1 **INTO** rowvar; --rowvar为行变量

FETCH curStudent **INTO** SID, Sname, sex;

请注意：游标的属性列必须与 目标列的数量一致，并且类型兼容。

注意：读取成功与否，可通过PL/SQL内置系统变量**FOUND**来判断

4.关闭游标

CLOSE cursorName;

当游标数据不再需要时，需要关闭游标，以释放其占有的系统资源，主要是释放游标数据所占用的内存资源，cursorName是游标名。

例如：**CLOSE** curStudent;

需要注意：当游标被关闭后，如果需要再次读取游标的数据，需要重新使用open打开游标，这时游标重新查询返回新的结果。

6.4.2 示例

1、下面例子使用不带参数的游标，查询student表的学号、学生姓名和性别。

```
CREATE OR REPLACE FUNCTION cursorDemo()
returns boolean as $BODY$
Declare
    unbound_refcursor refcursor; --定义变量及游标
    vsid varchar; --声明游标变量
    vsname varchar; --学号变量
    vsgender varchar; --姓名变量
    --性别变量
begin --函数开始
    open unbound_refcursor for execute 'select sid,sname,sex from student'; --打开
    loop --开始循环
        fetch unbound_refcursor into vsid,vsname,vsgender; --从游标中取值给变量
        if found then --检查从游标中是取到数据
            raise notice '%,%,% ',vsid,vsname,vsgender;
        else
            exit;
        end if;
    end loop; --结束循环
    close unbound_refcursor; --关闭游标
    raise notice '取数据循环结束...'; --打印消息
    return true; --为函数返回布尔值
    exception when others then raise exception 'error(%)',sqlerrm;-- sqlerrm错误代码变量,抛出异常
end; | --结束
$BODY$ LANGUAGE plpgsql; --规定语言
```

示例2:

2、下面示例使用带参数的游标，从成绩表中查询分数大于某给定值的学号和课程号。

```
create or replace function cursorDemo2(myscore int)
returns void as
$$
    declare
        vstuscore stu_score%ROWTYPE; --定义与表stu_score结构相同的行变量
        --定义带有一个输入参数的游标
        vstucursor cursor( invalue int)
        for select sid,cid,score from stu_score where score>=invalue order by sid;
    begin
        open vstucursor(myscore); --从外部传入参数给游标
        loop fetch vstucursor into vstuscore;
            exit when not found; -- 假如没有检索到数据，结束循环处理
            raise notice '%,%,%',vstuscore.sid,vstuscore.cid,vstuscore.score;
        end loop;
        close vstucursor; --关闭游标
    end;
$$ language plpgsql;

select cursorDemo2(85);
```

注意这两个例子里面的found 和 not found的用法

6.5 存储过程

- 把经常被重复使用的S Q L语句逻辑块封装起来，经编译后，存储在数据库服务器端；
- 当被再次调用时，而不需要再次编译；

6.5.1创建存储过程

2、创建存储过程的语法

```
CREATE OR REPLACE PROCEDURE procedureName([IN | OUT | INOUT] Pname dataType, ... )
AS $$
DECLARE
    变量1 数据类型 := 初始值1;
    变量2 数据类型 := 初始值2;
    .....
BEGIN
    -- 程序执行体语句写在这里;
END;
$$ LANGUAGE plpgsql //指明存储过程结束，告诉编译器使用PL/pgSQL语言实现的
```

- (1) **procedureName**: 存储过程名；
- (2) **OR REPLACE**: 覆盖同名的存储过程；
- (3) **IN、OUT或INOUT**参数模式。IN为输入参数；OUT为输出参数缺省值是IN。
- (4) **Pname**: 形式参数的名字。
- (5) **dataType**: 该存储过程参数的数据类型。

示例:

```

CREATE OR REPLACE PROCEDURE public.transfer(
    IN sender integer,
    IN receiver integer,
    IN amount numeric)
LANGUAGE 'plpgsql'
AS $BODY$
declare
    row_num int :=0;
    send_user accounts.username%type; --转出账户的姓名
    rec_user accounts.username%type; --转入账户的姓名
begin --转账事务开始
    update accounts set balance = balance - amount
    where id = sender and balance >= amount and amount>0;--余额减去转账金额
    GET DIAGNOSTICS row_num := ROW_COUNT;
    if row_num = 0 then --余额减转账金额不成功
        RAISE NOTICE '账户余额不足,或账户错误,请核对!';
        rollback;
    else
        update accounts set balance = balance + amount
        where id = receiver; --余额加上转账金额
        GET DIAGNOSTICS row_num := ROW_COUNT;

        if row_num = 0 then --余额加转账金额不成功
            RAISE NOTICE '账户错误,转账不成功!';
            rollback;
        else
            select username into send_user from accounts where id = sender;
            select username into rec_user from accounts where id = receiver;
            insert into translists(sendid,sendname,recid,recname,transdate,transamount)
                values(sender,send_user,receiver,rec_user,now(),amount);
            --插入转账信息,以备查帐使用
            GET DIAGNOSTICS row_num := ROW_COUNT;
            if row_num = 0 then
                RAISE NOTICE '转账登记出错,转账不成功!';
                rollback;
            else
                RAISE NOTICE '%向%成功转账%元!',send_user,rec_user,amount;
                commit;
            end if;
        end if;
    end if;
end
$BODY$;

```

6.5.2 修改存储过程

(1) 修改存储过程的名字

```
ALTER PROCEDURE name ( [ [ argmode ] [ argname ] argtype [, ...] ] )  
RENAME TO new_name;
```

(2) 修改存储过程的所有者

```
ALTER PROCEDURE name ( [ [ argmode ] [ argname ] argtype [, ...] ] )  
OWNER TO new_owner;
```

(3) 修改存储过程所属模式

```
ALTER PROCEDURE name ( [ [ argmode ] [ argname ] argtype [, ...] ] )  
SET SCHEMA new_schema;
```

把名字为displayStudent(integer)，参数类型为integer的存储过程名改为display_stu；再将所有者修改为dba；然后把所属模式改变为stuDB。

```
ALTER PROCEDURE displayStudent(integer) RENAME TO display_stu;  
ALTER PROCEDURE display_stu(integer) OWNER TO dba;  
ALTER PROCEDURE display_stu(integer) SET SCHEMA stuDB;
```

6.5.3 删除存储过程

示例：假如需要删除前面定义的存储过程 **TRANSFER**

```
DROP PROCEDURE IF EXISTS TRANSFER(SENDER int,RECEIVER int,AMOUNT dec);
```

6.5.4 存储过程并发编程

- ① **FOR UPDATE**：用于排他锁定，防止其他事务同时修改或锁定相同的数据。在SELECT语句中使用FOR UPDATE子句来实现。
- ② **FOR SHARE**：用于共享锁定，允许多个事务同时读取相同的数据，但阻止其他事务对数据进行更新。在SELECT语句中使用FOR SHARE子句来实现。
- ③ **FOR NO KEY UPDATE**：这种锁与FOR UPDATE类似，但它不会阻止其他事务对非键列进行更新。在SELECT语句中使用FOR NO KEY UPDATE子句来实现。
- ④ **FOR KEY SHARE**：用于共享锁定，允许多个事务同时读取相同的数据，但阻止其他事务对键列进行更新。在SELECT语句中使用FOR KEY SHARE子句来实现。

创建名为buy_train_ticket的PL/pgSQL过程，该函数将模拟购票操作，并使用并发控制：

```
Query  Query History
1  CREATE OR REPLACE PROCEDURE buy_train_ticket(p_ticket_id INTEGER, p_seats_to_buy INTEGER)
2  AS $$
3  DECLARE
4      v_available_seats INT;
5  BEGIN
6      --用for update 锁住火车票记录
7      SELECT available_seats INTO v_available_seats FROM train_tickets
8      WHERE id = p_ticket_id FOR UPDATE;
9      -- 检查是否有足够的座位数
10     IF v_available_seats < p_seats_to_buy THEN
11         RAISE EXCEPTION 'Not enough available seats';
12     END IF;
13     -- 更新座位数
14     UPDATE train_tickets SET available_seats = available_seats - p_seats_to_buy
15     WHERE id = p_ticket_id;
16     RAISE NOTICE 'Train tickets bought successfully.';
17 END;
18 $$ LANGUAGE plpgsql;
```

62

6.6 触发器

- 特殊类型的存储过程
- 与表相关联：必须定义在表或视图上
- 自动触发：由执行INSERT、DELETE、UPDATE操作时触发
- 不能直接调用，也不能传递或接受参数
- 是事务的一部分

6.6.1 触发器的分类

1.按触发的语句分为：

INSERT触发器、DELETE触发器、UPDATE触发器

2.按触发器执行的次数分为：

触发器执行的次数可分：

(1) 语句级触发器：由关键字**FOR EACH STATEMENT**声明，在触发器作用的表上执行一条SQL语句时，该触发器**只执行一次**，即使是修改了零行数据的SQL，也会导致相应的触发器执行。如果都没有被指定，**FOR EACH STATEMENT**会是**默认值**。

(2) 行级触发器：由关键字**FOR EACH ROW**标记的触发器，当触发器作用的表的数据发生变化时，**每变化一行就会执行一次触发器**。例如，假设学生成绩表有DELETE触发器，当在该表执行DELETE语句删除记录时，如果删除了20条记录，则将导致DELETE触发器被执行20次。

3.按触发的时间分为：

按触发的时间分为三类：

(1) **BEFORE触发器**：在触发事件之前执行触发器。

(2) **AFTER触发器**：在触发事件之后执行触发器。

(3) **INSTEAD OF触发器**：当触发事件发生后，执行触发器中指定的函数，而不是执行产生触发事件的SQL语句，从而替代产生触发事件的SQL操作。在表或视图上，对于INSERT、UPDATE或DELETE三种触发事件，每种最多可以定义一个INSTEAD OF触发器

6.6.2 触发器相关的特殊变量

(1) NEW

数据类型是RECORD。对于行级触发器，它存有INSERT或UPDATE操作产生的新的数据行。对于语句级触发器，它的值是NULL。

(2) OLD

数据类型是RECORD。对于行级触发器，它存有被UPDATE或DELETE操作修改或删除的旧的数据行。对于语句级触发器，它的值是NULL。

(3) TG_OP

数据类型是text；是值为INSERT、UPDATE、DELETE的一个字符串，它说明触发器是为哪个操作引发。

6.6.3 创建触发器

CREATE TRIGGER 触发器名

{ **BEFORE | AFTER | INSTEAD OF** }

ON 表名

[**FOR | EACH**] { **ROW | STATEMENT** }]

EXECUTE PROCEDURE 存储过程名 (参数列表)

(1) 指明所定义的触发器名

(2) **BEFORE | AFTER | INSTEAD OF** 指明触发器被触发的时间

(3) **ON** 表名 指明触发器所依附的表

(4) **FOR EACH** { **ROW | STATEMENT** } 指明触发器被触发的次数

(5) **EXECUTE PROCEDURE** 存储过程名 (参数列表) 指明触发时所执行的存储过程

、创建触发器的基本步骤：

- ① 检查数据库中将要创建的触发器所依附的表或视图是否存在，如果不存在，必须首先创建该表或视图。
- ② 创建触发器被触发时所执行的触发器函数，该函数的类型必须是**TRIGGER**型，是触发器的执行函数。但要注意，有些数据库不需要独立定义触发器函数，而是在创建触发器时，定义触发器的过程体。
- ③ 创建触发器，一般需要指明触发器依附的表，触发器被触发执行的时间，触发器是行级触发器还是语句级触发器，触发器执行需要满足的条件。

现在创建触发器函数score_audit()

```
CREATE OR REPLACE FUNCTION score_audit() RETURNS TRIGGER AS $score_audit$
BEGIN
    IF (TG_OP = 'DELETE') THEN
        INSERT INTO audit_score SELECT user, old.sid, old.cid, now(), OLD.score ;
        RETURN OLD;

    ELSIF (TG_OP = 'UPDATE') THEN
        INSERT INTO audit_score
        SELECT user, old.sid, old.cid, now(), OLD.score , new.score
        where old.sid=new.sid and old.cid=new.cid;
        RETURN NEW;

    ELSIF (TG_OP = 'INSERT') THEN
        INSERT INTO audit_score SELECT user, new.sid, new.cid, now(),null, new.score;
        RETURN NEW;
    END IF;
    RETURN NULL;
END;
$score_audit$ LANGUAGE plpgsql;
```

接下来在student_score表上创建触发器score_audit_triger

CREATE TRIGGER score_audit_triger

AFTER INSERT OR UPDATE OR DELETE ON student_score

FOR EACH ROW EXECUTE PROCEDURE score_audit();

注意：

- for each row
- after
- INSERT OR UPDATE OR DELETE
- procedure

6.6.4 触发器的修改、删除

ALTER TRIGGER name **ON** table_name **RENAME TO** new_name

主要参数说明：

- (1) name: 需要修改的现有触发器的名称。
- (2) table_name: 该触发器作用的表的名字。
- (3) new_name: 现有触发器的新名字。

例如：将上述定义的触发器改名为score_audit_trig

ALTER TRIGGER score_audit_trigger **ON** student_score **RENAME TO** score_audit_trig;

DROP TRIGGER [IF EXISTS] name **ON** table_name

主要参数说明：

- (1) IF EXISTS: 如果指定的触发器不存在，那么发出提示而不是抛出错误；
- (2) name: 要删除的触发器名；
- (3) table_name: 触发器定义所依附的表的名称。

例如：将上述触发器score_audit_trig删除，同时级联删除依赖触发器的对象。

DROP TRIGGER IF EXISTS score_audit_trig **ON** student_score ;

6.6.5 触发器示例

示例1：



示例2：

在电商系统中，针对订单 **orders**、订单详情 **order_items**、产品库存 **products.stock** 之间的约束关系，请编写触发器，**实现在插入订单详情和更新订单详情时，检查库存数量是否满足**；当库存数量满足时，才更新或插入订单详情，同时更新订单总金额和库存数量，如果库存不满足要求，则阻止插入或更新订单详情。

```
CREATE OR REPLACE FUNCTION check_and_update_order()
RETURNS TRIGGER AS $$
DECLARE
    current_stock INT;
BEGIN
    -- 获取当前库存
    SELECT stock INTO current_stock FROM products WHERE product_id = NEW.product_id;

    -- 检查库存是否足够
    IF current_stock < (COALESCE(NEW.quantity,0)-COALESCE(OLD.quantity,0)) THEN
        RAISE EXCEPTION '库存不足，无法插入或更新订单详情';
    END IF;

    -- 更新库存数量
    UPDATE products
    SET stock = stock - (COALESCE(NEW.quantity,0)-COALESCE(OLD.quantity,0))
    WHERE product_id = NEW.product_id;

    -- 更新订单的总金额
    UPDATE orders
    SET total_amount = total_amount+(COALESCE((NEW.quantity * NEW.price),0)-COALESCE((OLD.quantity * OLD.price),0))
    WHERE order_id = NEW.order_id;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

*插入的旧值 OLD 是 null
→ 即处理了插入又处理了更新*

② 当插入时，OLD.quantity=0；当修改时，括号里计算quantity的变化量，函数COALESCE将null转为0；null与任何数进行计算都得null，而非0。

③ total_amount 加上订单金额的变化量，得到新的金额

*这里用的 before
所以不能用之前的 sum 了
因为还没更新*

Output Messages Notifications

FE FUNCTION

returned successfully in 51 msec.

Query Query History

```
1 -- 创建触发器
2 CREATE TRIGGER trg_check_and_update_order
3 BEFORE INSERT OR UPDATE ON order_items
4 FOR EACH ROW
5 EXECUTE FUNCTION check_and_update_order();
```

② BEFORE类型触发器

③ 必须是行级触发器

④ 指定触发器函数

6.6.6 事件触发器

- 事件触发器是定义在数据库级，权限相对较大，所以只有超级用户才能创建和修改触发器

- ① **ddl_command_start**：在DDL开始前触发；
- ② **ddl_command_end**：在DDL结束后触发；
- ③ **sql_drop**：删除数据库对象前被触发，其中删除的数据库对象详细信息，可以通过 **pg_event_trigger_dropped_objects()** 函数记录下来。

2、DDL操作会触发的事件列表

DDL关键字	ddl_command_start	ddl_command_end	sql_drop
CREATE	√	√	-
ALTER	√	√	-
DROP	√	√	√
SELECT INTO	√	√	-

与普通规则触发一样，用户必须声明在事件触发器被触发时所要执行的函数，该函数不带参数的函数并且返回 `event_trigger` 类型，返回类型只是作为事件触发器调用该函数的消息。

定义事件触发器，一般需要指明触发事件触发器的事件类型，事件触发器执行的筛选条件，事件触发被触发时执行的过程函数。

CREATE EVENT TRIGGER name ON event

[WHEN filterVariable IN (filterValue [, ...]) [AND ...]]

EXECUTE PROCEDURE functionName();

- ① **name**: 定义的新触发器的名字。这个名字必须在数据库内是唯一的。
- ② **event**: 触发调用触发器函数的事件名字。
- ③ **filterVariable**: 筛选事件的变量名称，指定它所支持的触发该触发器的事件子集TAG；
- ④ **filterValue**: 可以触发该触发器的 filterVariable 相关的值，指定TAG所限定的命令列表，如：('DROP FUNCTION', 'CREATE TABLE')。
- ⑤ **function_name**: 用户声明的不带参数的函数，返回 `event_trigger` 类型。

- 例如：创建名字为NoCreateDrop的事件触发器，禁止用户postgres执行CREATE TABLE和DROP TABLE命令。如果用户postgres执行该命令，抛出异常提示信息。

第1步：用plpgsql语言创建事件处理函数abort()

The screenshot shows a PostgreSQL IDE interface. On the left is the 'Object Explorer' showing the database structure. The main window displays a SQL query editor with the following code:

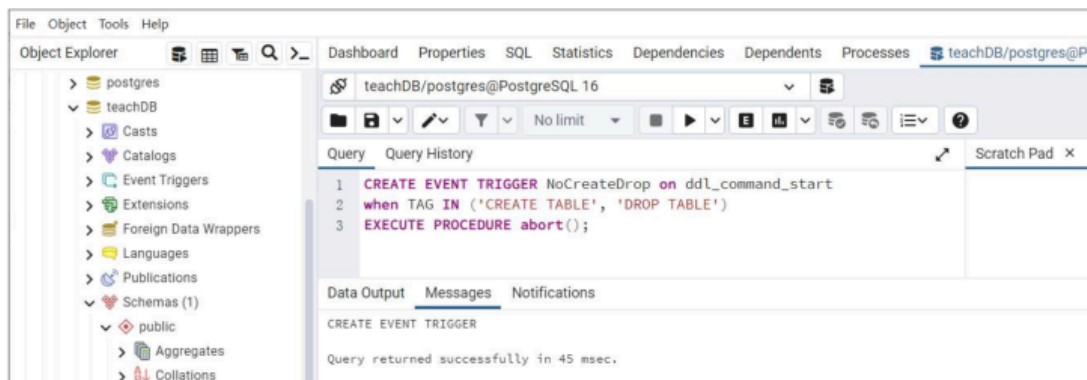
```
1 CREATE OR REPLACE FUNCTION abort()
2 RETURNS event_trigger
3 AS $$
4 BEGIN
5     if current_user = 'postgres' then
6         RAISE EXCEPTION 'event:%, command:%', tg_event, tg_tag;
7     end if;
8 END;
9 $$ LANGUAGE plpgsql;
```

Below the query editor, there is a 'Data Output' section showing the message: 'Query returned successfully in 62 msec.'.

Two annotations are present on the right side of the screenshot:

- > `tg_event`: 为 `ddl_command_start`, `ddl_command_end`, `sql_drop`;
- > `tg_tag`: 实际执行的DDL操作, 如: `create table`, `drop table`等。

第2步：创建事件触发器NoCreateDrop。



4、修改事件触发器

使用 **ALTER EVENT TRIGGER** 改变现有的事件触发器的定义。但执行该语句的用户必须是超级用户才能修改事件触发器。

- ① **ALTER EVENT TRIGGER name DISABLE;** //禁用已有的触发器
- ② **ALTER EVENT TRIGGER name ENABLE;** //使该事件触发器激活
- ③ **ALTER EVENT TRIGGER name OWNER TO new_owner;** //改变触发器拥有者
- ④ **ALTER EVENT TRIGGER name RENAME TO new_name;** //改变触发器的名字

5、删除事件触发器

使用 **DROP EVENT TRIGGER**删除已存在的事件触发器。

其语法格式如下：

DROP EVENT TRIGGER [IF EXISTS] name;

例如：删除事件触发器**NoCreateDrop**。

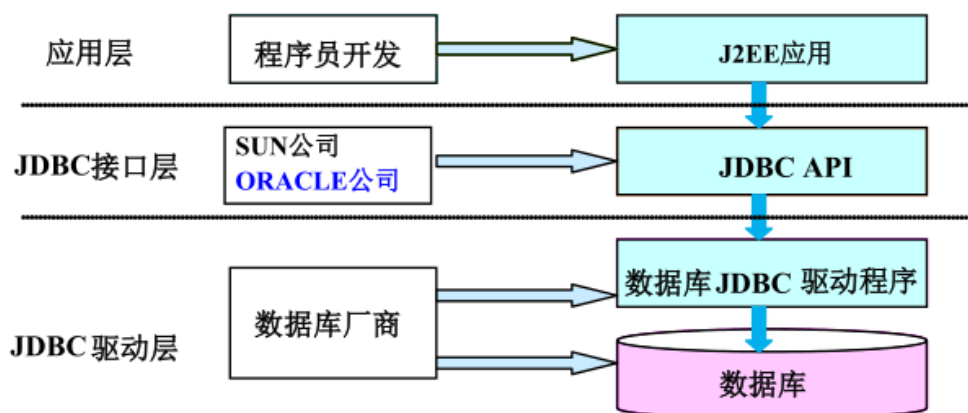
DROP EVENT TRIGGER IF EXISTS NoCreateDrop;

6.7 应用程序编程访问数据库

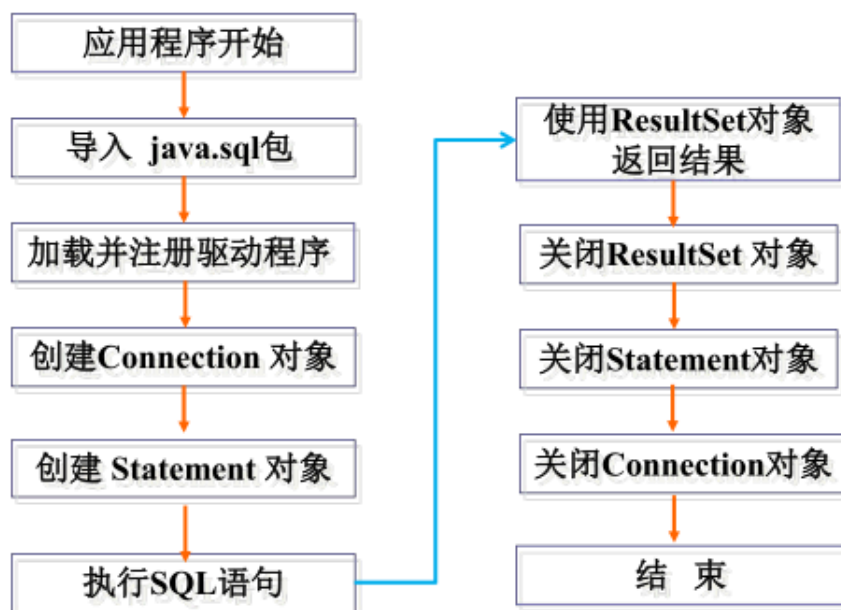
使用JDBC访问数据库

- 由一组用Java编程语言编写的类和接口组成。这个API由**java.sql.*包**中的一些类和接口组成

1、JDBC工作原理



2、JDBC程序访问数据库步骤



(1) 加载驱动

① 加载JDBC驱动是通过调用方法：

`Class.forName("驱动名字")`

② PostgreSQL数据库驱动程序加载语句：

`Class.forName("org.postgresql.Driver");`

(2) 建立连接

- 与数据库建立连接的方法是调用**DriverManager.getConnection(String url)**方法。
- 下面与PostgreSQL数据库建立连接的语句：
 - ① `String URL = "jdbc:postgresql://localhost:5432/testdb";`
 - ② `String userName = "myuser";`
 - ③ `String passWord = "sa";`
 - ④ `connection conn = DriverManager.getConnection(URL,userName,passWord);`

(3) 创建Statement对象

- 可以用Connection对象的方法**createStatement()**创建Statement。
示例如下：

```
Statement stmt = conn.createStatement();
```

(4) 执行SQL语句

- ① 创建了Statement对象,就可以向Statement对象发送SQL语句。主要掌握两种执行SQL语句的方法：**executeQuery()**、**executeUpdate()**
- ② **executeQuery()**：返回语句执行后的**单个结果集**的，所以通常用于**select语句**
- ③ **executeUpdate()**返回值是一个整数，指示受影响的行数（可以用于**update**、**insert**、**delete**语句）。

示例如下：

```
Statement stmt = conn.createStatement();  
String sql = "INSERT INTO public.student (sid, sname, gender, birthday, major, phone)"  
            + " VALUES ('2017001', '张山', '男', '1998-10-10', '软件工程', '13602810001')";  
stmt.executeUpdate(sql);
```

(5) ResultSet保存结果集

- ResultSet对象被称为**结果集**，代表符合SQL语句条件的所有行，并且它通过一套**getXXX方法**提供了对这些行中数据的访问。
- ResultSet里的数据一行一行排列，每行有多个字段，并且有一个记录指针，指针所指的数据行叫做**当前数据行**，我们只能来操作当前的数据行。如果**想要取得某一条记录**，就要使用ResultSet的**next()方法**，如果我们**想要得到ResultSet里的所有记录**，就应该**使用while循环**。

示例代码如下：

```
Statement stmt = conn.createStatement();
String sql = "SELECT id, name, age FROM company";
ResultSet rs = stmt.executeQuery(sql);
```

(6) 关闭连接

作为一种好的编程风格，在不需要**ResultSet对象**、**Statement对象**和**Connection对象**时，应该**显式地关闭**它们。关闭这些对象的方法为：**close()**；

示例：

- ① `rs.close();` //关闭结果集对象
- ② `stmt.close();` //关闭执行对象
- ③ `conn.close();` //关闭连接对象

1、Java使用JDBC连接数据库的例子

```
1 package testConnDB;
2 import java.sql.Connection;
3 import java.sql.DriverManager;
4 import java.sql.ResultSet;
5 import java.sql.Statement;
6
7 public class JDBCdemoDB {
8     public static void main( String args[] )
9     {
10         Connection conn = null;
11         Statement stmt = null;
12         String URL = "jdbc:postgresql://localhost:5432/testDB";
13         String userName = "myuser";
14         String password = "sa";
15
16         try {
17             Class.forName("org.postgresql.Driver");
18             conn = DriverManager.getConnection(URL , userName, password );
19             System.out.println("Opened database successfully");
20         }
```

```

String sql = "select * from student"; // 查询数据的sql语句
stmt = (Statement) conn.createStatement(); //创建执行sql语句的Statement对象, stmt属局部变量

ResultSet rs = stmt.executeQuery(sql); //执行sql查询语句, 返回查询数据的结果集
System.out.println("最后的查询结果为: ");
while (rs.next()) { // 判断是否还有下一个数据

    //输出查到的记录的各个字段的值
    System.out.println(rs.getString("sid") + " " + rs.getString("sname") + " "
        + rs.getString("sex") + " " + rs.getString("classid") );
}

stmt.close();
conn.close();
} catch ( Exception e ) {
    System.err.println( e.getClass().getName()+" : "+ e.getMessage() );
    System.exit(0); }
}
}

```

嵌入式SQL数据库访问编程

看PPT吧, P136开始

有些点, 还是再看看

- 1) `ResultSet.next()`将游标由当前位置移动到下一行;
 - 2) `ResultSet.getString(String columnName)` 获取指定字段的String类型值;
 - 3) `ResultSet.getString(int columnIndex)` 获取指定索引的String类型值;
 - 4) `ResultSet.previous()`将游标由当前位置移动到上一行。
-
- 1) `prepareStatement.executeUpdate()`执行更新;
 - 2) `prepareStatement`对象使用`addBatch()`向批处理中加入更新语句,
 - 3) `executeBatch()`方法用于成批地执行SQL语句。

```

package testConnDB;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;

public class SQLInJava {

    public static void main(String[] args) {
        Connection conn = null;
        String URL = "jdbc:postgresql://localhost:5432/testDB";
        String userName = "myuser";
        String password = "sa";
        String sid[] = {"14102", "14103", "14202", "14301", "14101", "14201", "14503"};
        String cid[] = {"1205", "1208", "1205", "1208", "1201", "1201", "1201"};
        int score[] = {90, 78, 89, 68, 86, 96, 83};
        try {
            Class.forName("org.postgresql.Driver");
            conn = DriverManager.getConnection(URL, userName, password);
            System.out.println("成功连接数据库!");

            String insertSql = "INSERT INTO stu_score(sid, cid, score) VALUES (?, ?, ?)";
            String querySql = "select sid, cid, score from stu_score where score >=?";
            PreparedStatement psInsert = conn.prepareStatement(insertSql); //定义动态执行SQL语句对象
            PreparedStatement psQuery = conn.prepareStatement(querySql); //定义动态执行SQL语句对象

```

```
for (int i=0; i<sid.length; i++)
{
    psInsert.setString(1, sid[i]);
    psInsert.setString(2, cid[i]);
    psInsert.setInt(3, score[i]);
    psInsert.addBatch();    //添加批处理的记录
}
psInsert.executeBatch();//批处理执行多条数据记录

psQuery.setInt(1, 80);
ResultSet rs = psQuery.executeQuery();
while (rs.next()) { // 判断是否还有下一个数据
    System.out.println(rs.getString("sid") + " " + rs.getString("cid") + " " + rs.getInt("score") );
}

psQuery.close();
psInsert.close();
conn.close();
} catch ( Exception e ) {
    System.err.println( e.getClass().getName()+" : "+ e.getMessage() );
    System.exit(0);
}
}
```

